

Large Language Model Foundations

Mengyue Yang

Natural Language Processing

Language is the most natural, profound, and convenient tool for humans to communicate ideas and express emotions.

“Language is the greatest evolutionary achievement since the eukaryotic cell.”
— **Edward O. Wilson**, the father of sociobiology

“Language itself is the greatest technological invention in human history.”
— **James Gleick**, *The Information*

Most of human knowledge throughout history has been recorded and passed down in the form of language and writing.

Natural Language Processing

- **Cognitive intelligence** is a crucial stage of artificial intelligence, enabling machines to understand, reason and think.
- **Natural Language Understanding (NLU)** is an essential component of cognitive intelligence, a necessary path to general artificial intelligence.

Computational Intelligence

Able to store and
calculate

Perceptual Intelligence

Able to hear, see, and
recognize

Cognitive Intelligence

Able to understand,
reason, and think

Natural Language Processing

- Natural Language Understanding, Generation

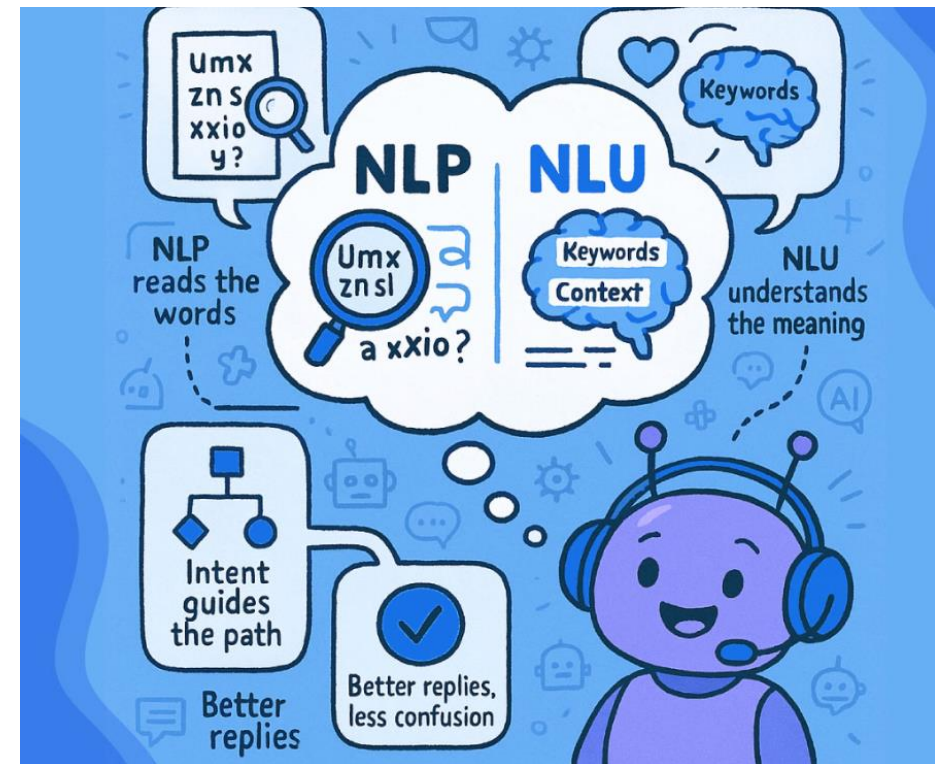


RAM
CPU
Price
...

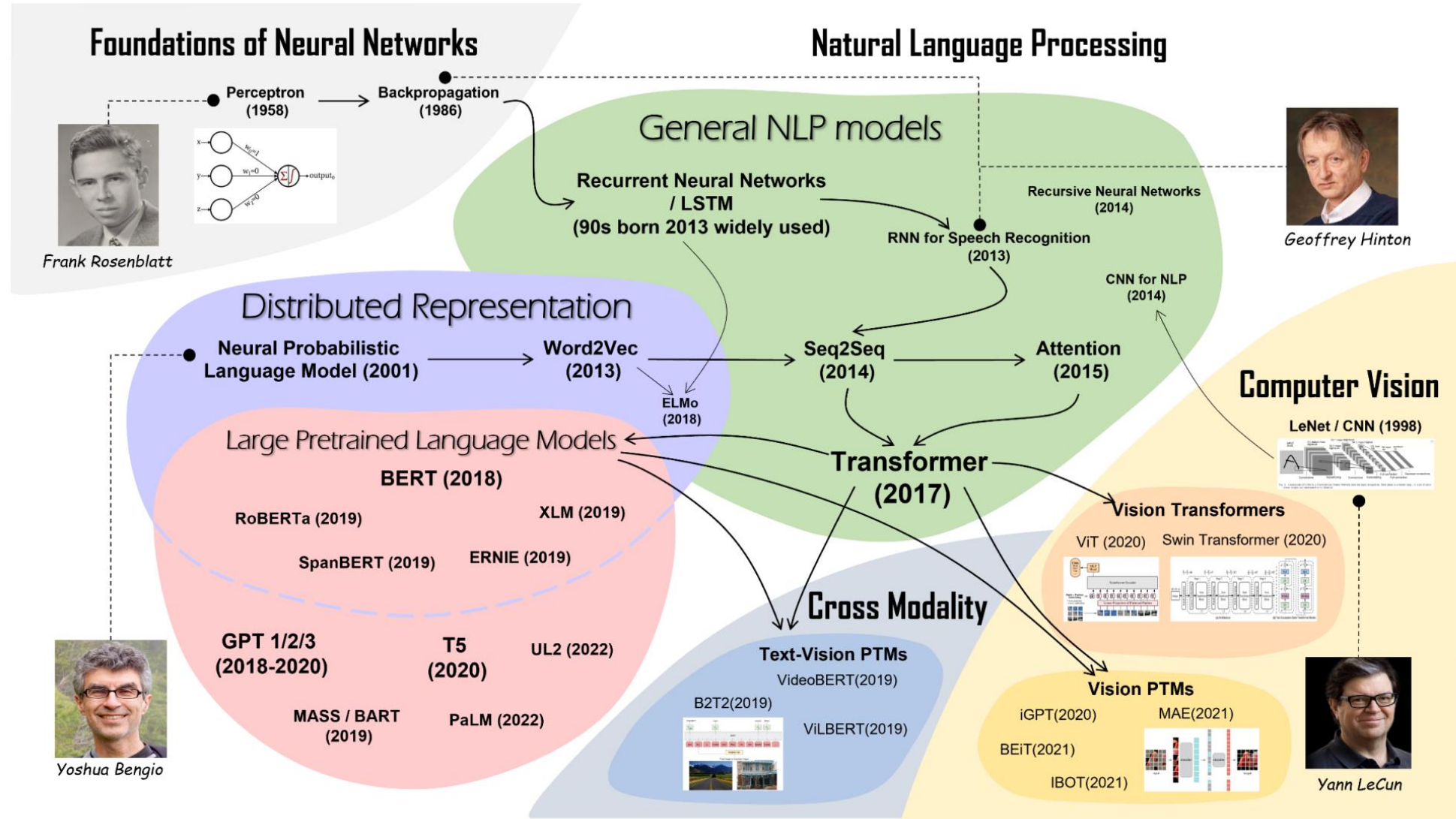
The **appearance** of the PC looks good, but the battery life is too short.



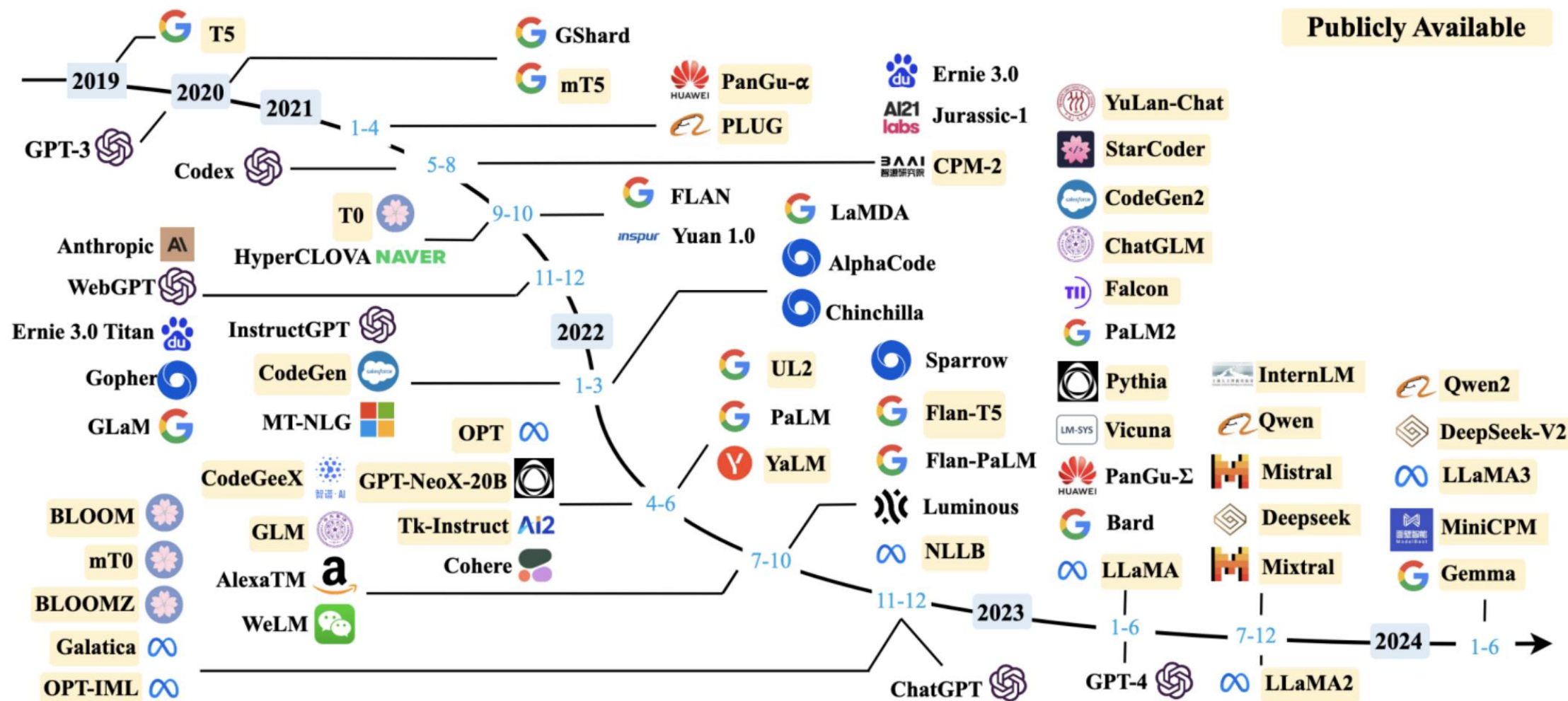
The **appearance** of this dress looks nice, and the fabric is not bad.



Natural Language Processing



Natural Language Processing



What is Language Models

- Language Model
 - Predicting the probability of a token in a sentence.

$$p(s) = p(w_1, w_2 \dots, w_m)$$

- Where does the difficulty of Natural Language Understanding (NLU) lie?

Language is a highly abstract construct; its basic building blocks are not well-defined physical quantities.

Statistical
Language Model
N-gram

Neural Language
Models
NNLM, word2vec

Pre-trained Language
Model
BERT, T5

Large Language
Model
GPT-3, GPT-4

Language Models

- Language Model
 - Predicting the probability of a token in a sentence.

S1: The essence of a language predict is to the probability model of a piece of natural-language text.

S2: The essence of a probability predict language model is to the of a piece of natural-language text.

S3: The essence of a language model is to predict the probability of a piece of natural-language text.

Which one is correct??

$$P(S) = P(w_1, w_2, \dots, w_n) = P(w_1)P(w_2|w_1)P(w_3|w_1, w_2) \cdots P(w_n|w_1, w_2, \dots, w_{n-1})$$

$$L = \sum_{w \in C} \log P(w|\text{context}(w))$$

Language Model: Modelling the probability a token in a sentence

Data Preprocess — Step1: Tokenization

- **Tokenization.**

- The process of splitting a given character sequence into a series of subsequences.
- Each subsequence is called a token.
- The main goal of tokenization is to determine the correct tokens and avoid interference from factors such as punctuation.

`texts[i] = "the cat sat on the bed."`



Tokenization

`tokens[i] = ["the", "cat", "sat", "on", "the", "bed"]`

Data Preprocess — Step1: Tokenization

- **Splitting words and phrases**
 - To be or not to be...
- **The impact of punctuation**
 - Hyphens: self-motivation
- **Apostrophes/quotes: Shaquille O'Neal ("Big Shaq")**
- **Splitting proper nouns**
 - New York University or New / York University?

Data Preprocess — Step1: Tokenization

- **Potential pitfalls of tokenization**

- Tokenization can create a large number of low-frequency tokens, leading to severe data sparsity and an increasing number of OOV (out-of-vocabulary) words.
- Inevitable errors and omissions in tokenization introduce additional noise.

- **Stopword removal:** stopwords are words that appear frequently in documents or have little impact on the actual meaning.

- For example, *the*, *of* in English
- Numbers, adverbs, and other terms weakly related to semantics are often treated as stopwords.

Data Preprocess — Step1: Tokenization

- **Stopword types and identification**

- Stopword selection depends on the nature of the corpus.

- For example, *www* in URLs; *wiki* in Wikipedia.

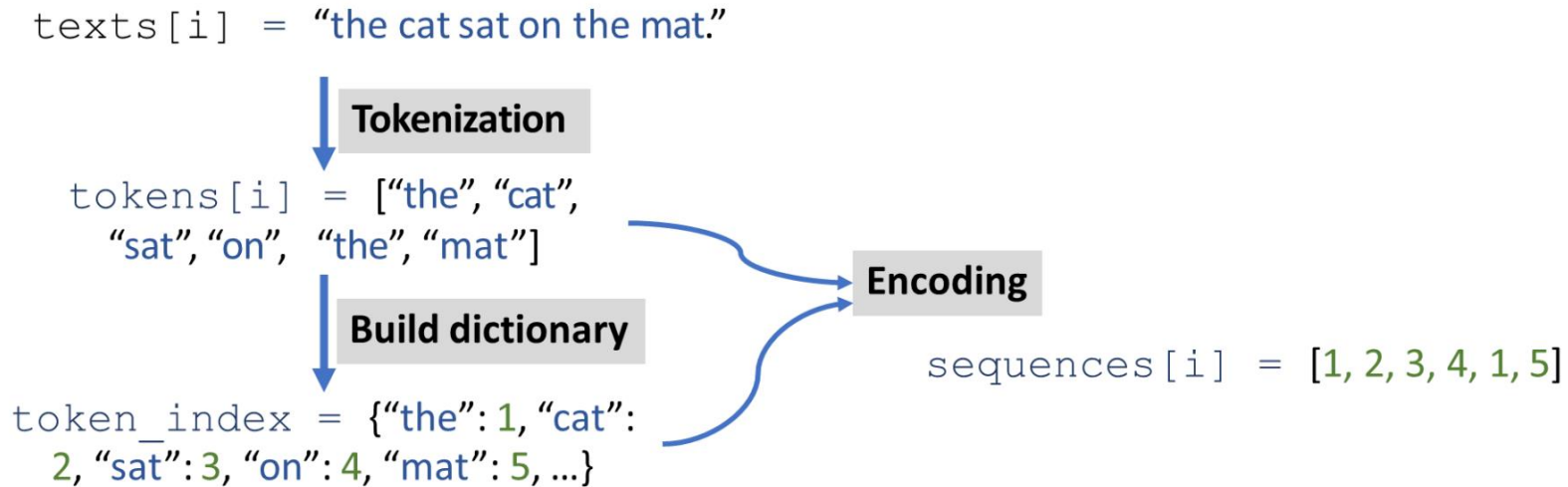
- Common stopwords identification methods:

- More mature approaches include text frequency analysis, word-frequency statistics, and entropy-based measures.

- Advanced stopwords identification methods:

Data Preprocess — Step 2: Dictionary

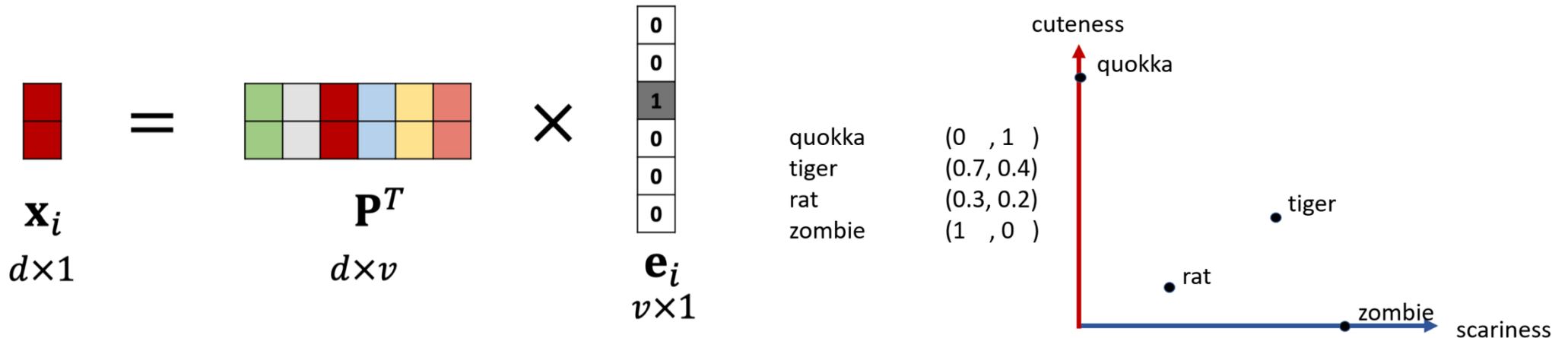
- **Dictionary**
 - Every word can use a number to present.



Data Preprocess — Step 3: Word Embedding

- **Word Embedding**

- To address the above issues, word embeddings map **one-hot vectors** into **low-dimensional vectors**.
- **D** is the embedding dimension, chosen by the user; the size of **D** affects the model's expressive power.
- The parameter matrix **P** is learned by the model during training.



Language Models

- Language Model
 - Predicting the probability of a token in a sentence.

S1: The essence of a language predict is to the probability model of a piece of natural-language text.

S2: The essence of a probability predict language model is to the of a piece of natural-language text.

S3: The essence of a language model is to predict the probability of a piece of natural-language text.

Which one is correct??

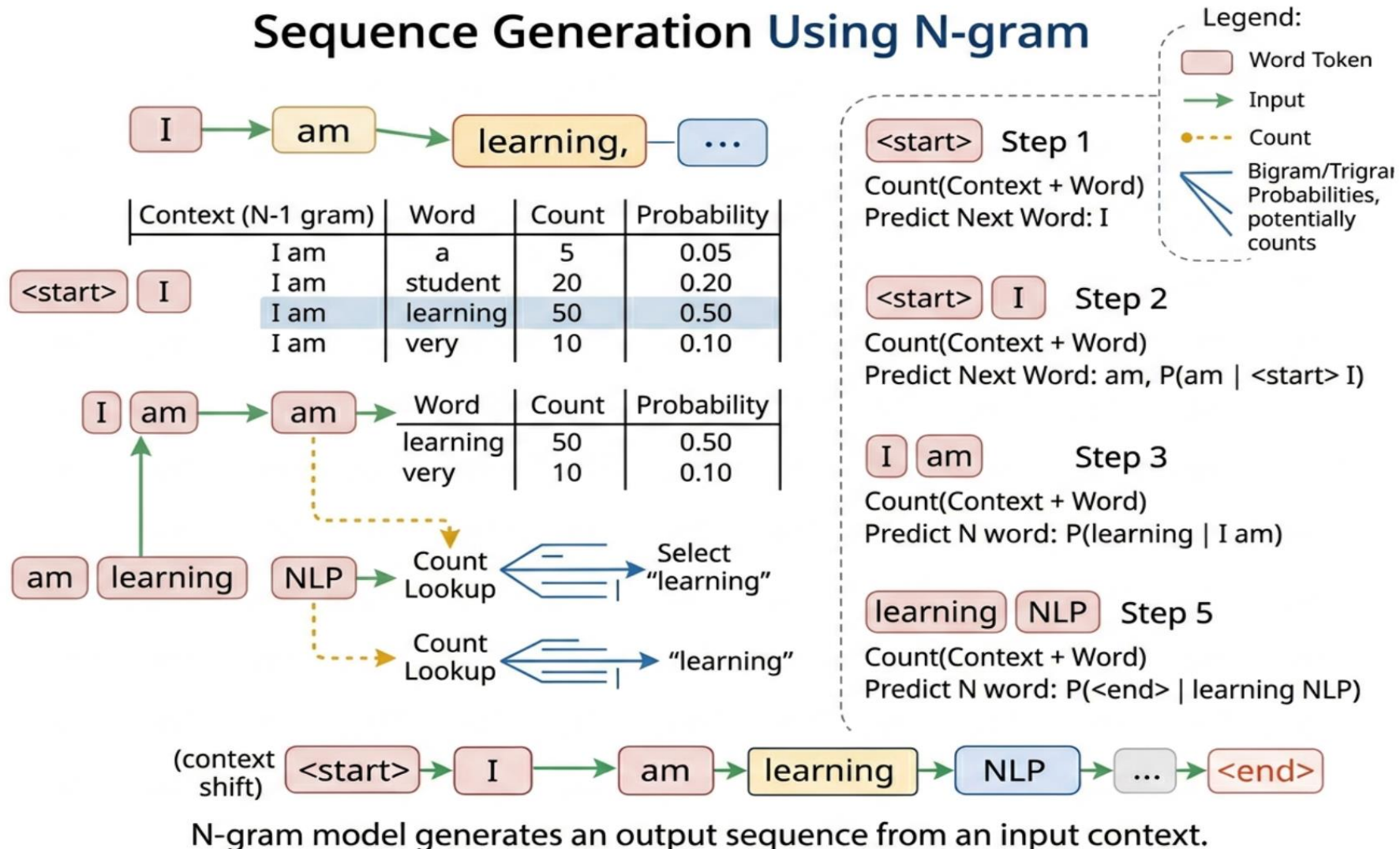
$$P(S) = P(w_1, w_2, \dots, w_n) = P(w_1)P(w_2|w_1)P(w_3|w_1, w_2) \cdots P(w_n|w_1, w_2, \dots, w_{n-1})$$

$$L = \sum_{w \in C} \log P(w|\text{context}(w))$$

Language Model: Modelling the probability a token in a sentence

Statistical Language Model — N-gram

Sequence Generation Using N-gram

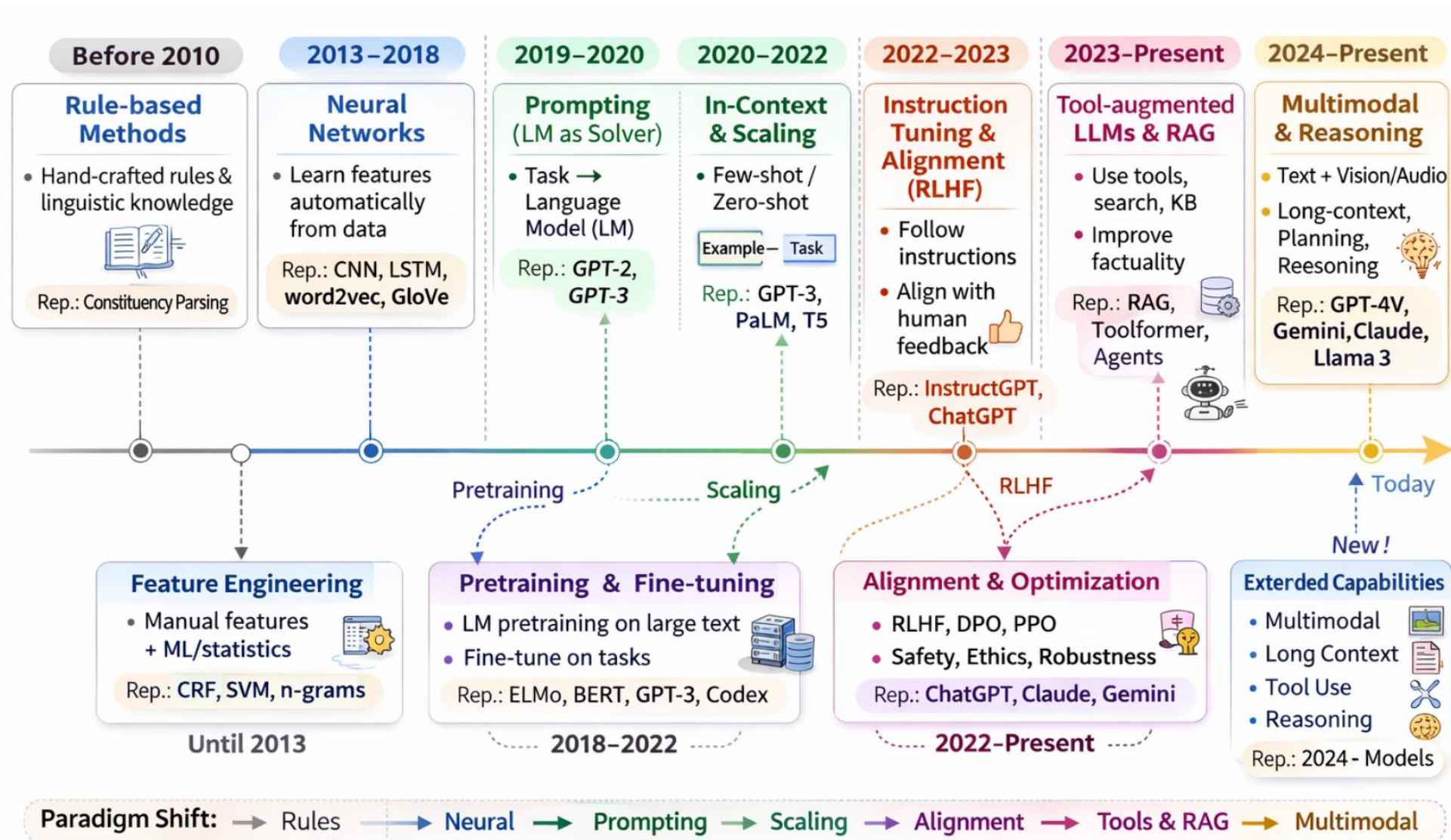


Statistical Language Model — N-gram

- **Problems of N-gram**

- Sparsity: if we didn't see it, then we can not predict it
- No semantic meaning
- Short view because of small context window

Language Models

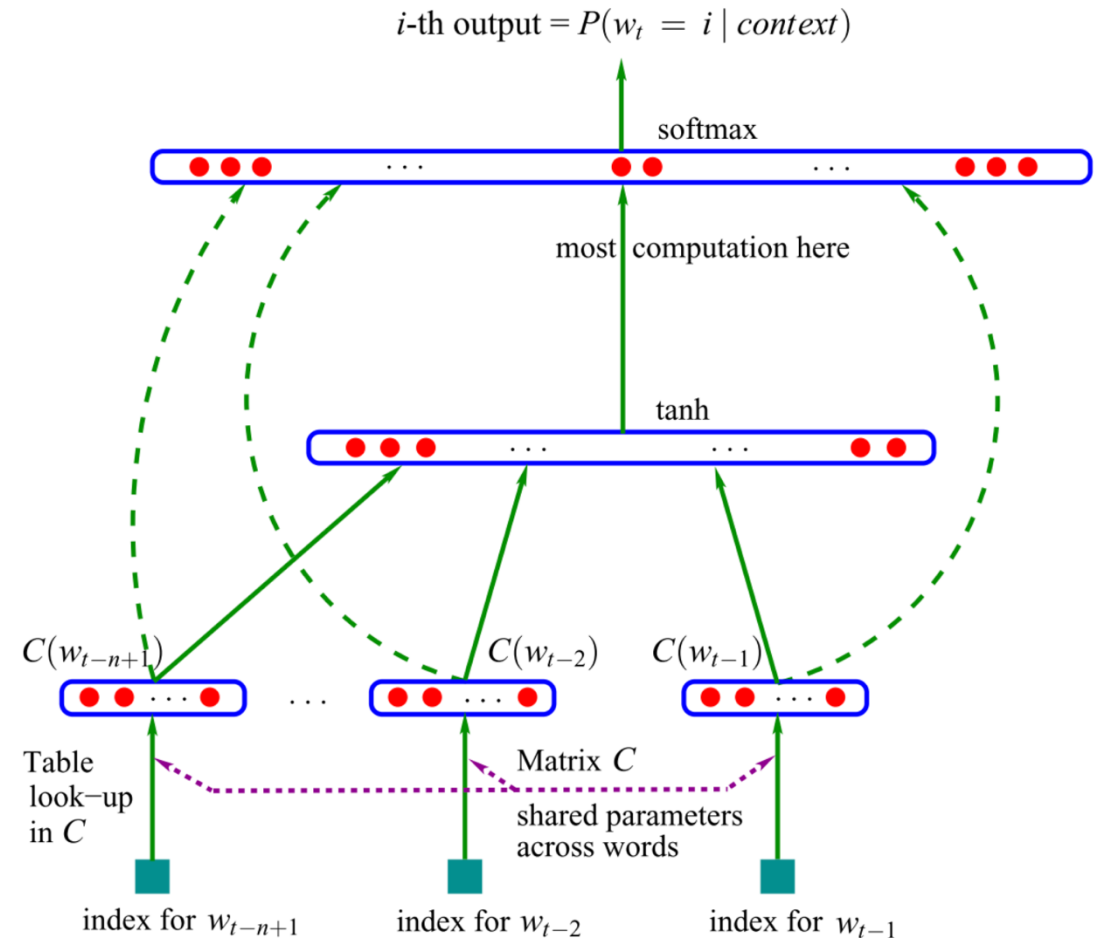


Language Models — Some milestones

- **NNLM (2001, Bengio)**

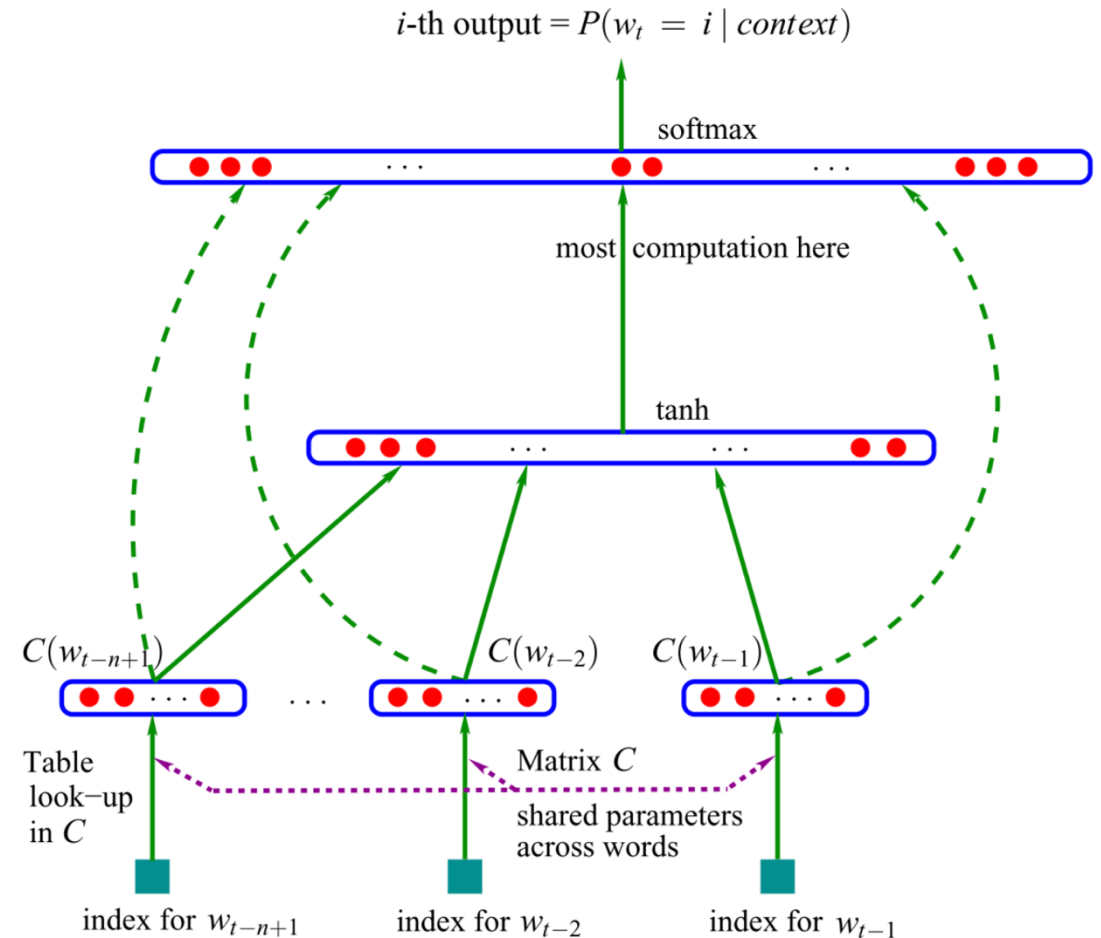
- Transform each word into a **mathematical vector containing specific features** (this is the prototype of Word Embeddings).
- Use a neural network to guess the next word.

- The vector for **Cat** might be $[0.9, 0.2, 0.9]$
- The vector for **Dog** might be $[0.8, 0.3, 0.9]$
- The vector for **Stone** might be $[0.0, 0.0, 0.0]$



Language Models — Some milestones

- **NNLM (2001, Bengio)** — A language model built using feedforward neural networks
 - NNLM consists of an **input layer**, a **hidden layer**, and an **output layer**.
 - The input layer is a matrix **C**. Each time a word is input, it corresponds to looking up the row in matrix **C** that represents that word.
 - This mechanism is the **prototype of word embeddings**.



Language Models — Some milestones

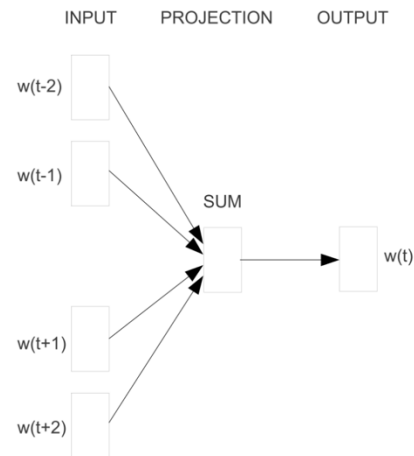
- **Word2Vec (2013)** is a super minimalist network **specifically designed to train word vectors**.
 - The meaning of a word is determined by who it frequently hangs out with

Suppose there is a fruit you've never seen before called "Gala fruit".
You read a few sentences:

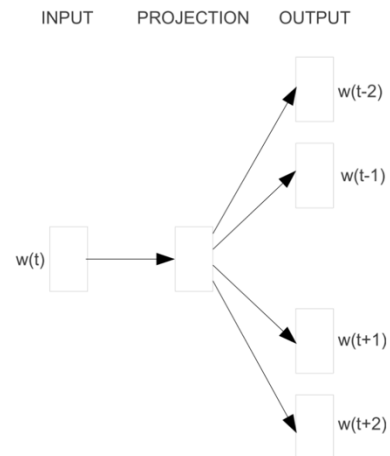
1. "I peeled a Gala fruit to eat."
2. "The juice of the Gala fruit is very sweet."
3. "The tree is full of red Gala fruits."

Language Models — Some milestones

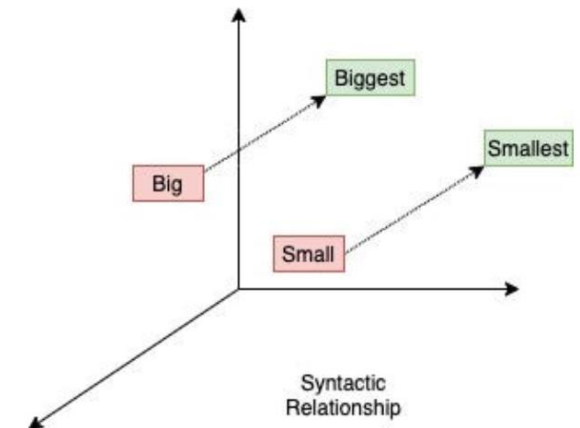
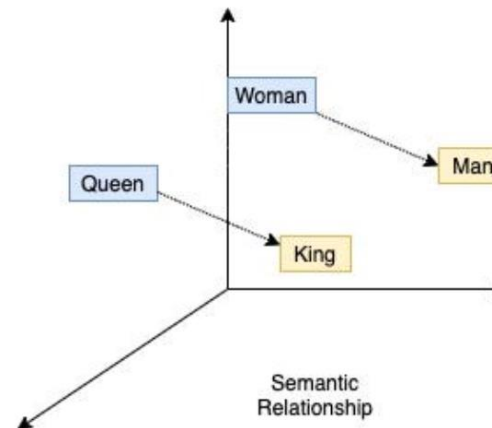
- **Word2Vec (2013)** Word2vec learns word embeddings by **predicting context**:
 - CBOW predicts a target word from its surrounding words, while Skip-gram predicts surrounding words from a target word.
 - The learned embeddings place **semantically similar words close together in vector space**.



CBOW



Skip-gram



Language Models — Some milestones

- **Word2Vec (2013)** Word2vec learns word embeddings by **predicting context: CBOW**

The Setup

- **The Sentence:** "The dog ____ loudly."
- **Target Word (Missing):** "barks"
- **Context Words (Clues):** "dog" and "loudly"
- **Our Tiny Vocabulary:** [dog, barks, loudly, apple]

Language Models — Some milestones

- **Word2Vec (2013)** How to predict:

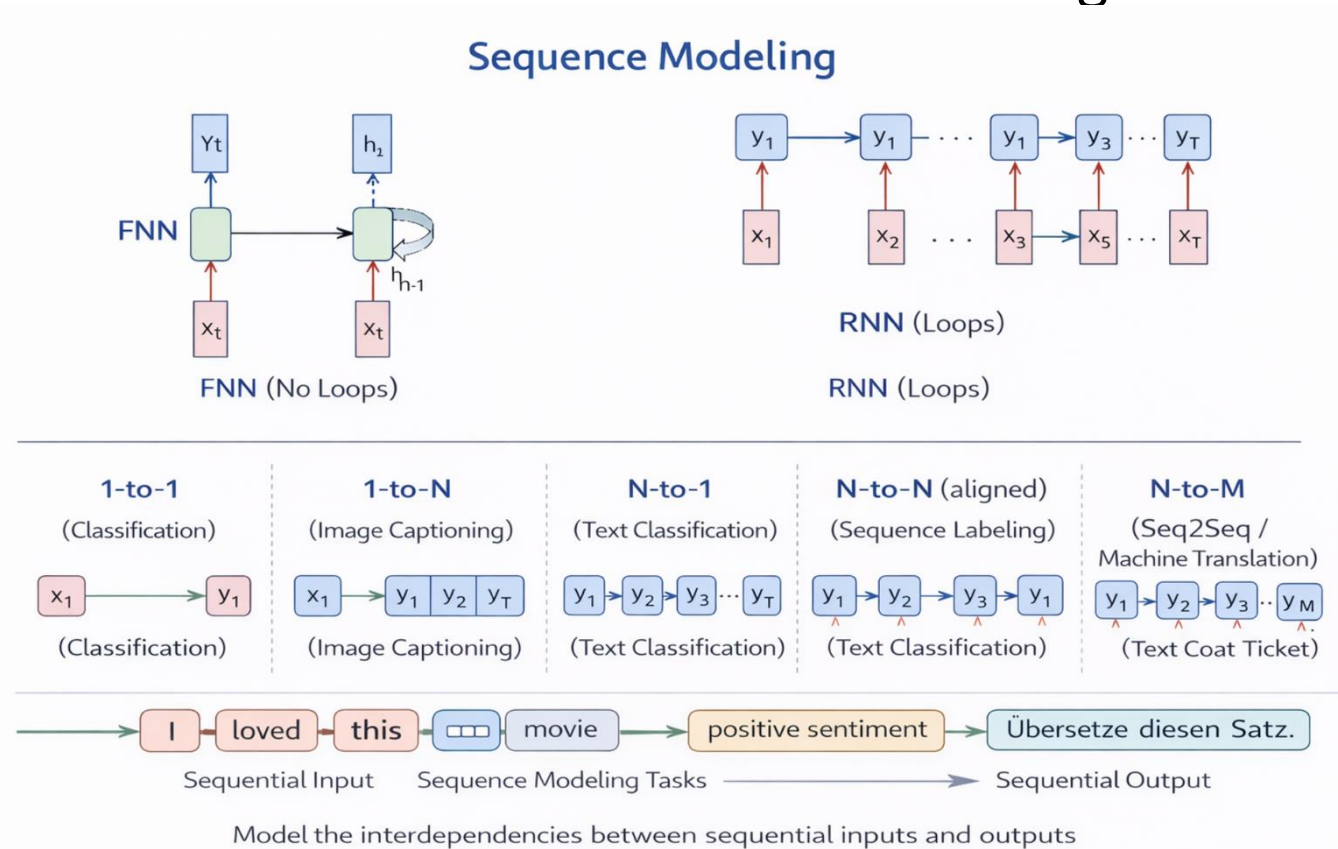
- Step 1: Look up the Vectors (Word Embeddings)
 - Vector for "dog" = [1.0, 0.2]
 - Vector for "loudly" = [0.4, 0.8]
- Step 2: Add and Average
 - Average = $([1.0, 0.2] + [0.4, 0.8]) / 2 = [0.7, 0.5]$
- Find the most similar word from embeddings

The Setup

- **The Sentence:** "The dog ____ loudly."
- **Target Word (Missing):** "barks"
- **Context Words (Clues):** "dog" and "loudly"
- **Our Tiny Vocabulary:** [dog, barks, loudly, apple]

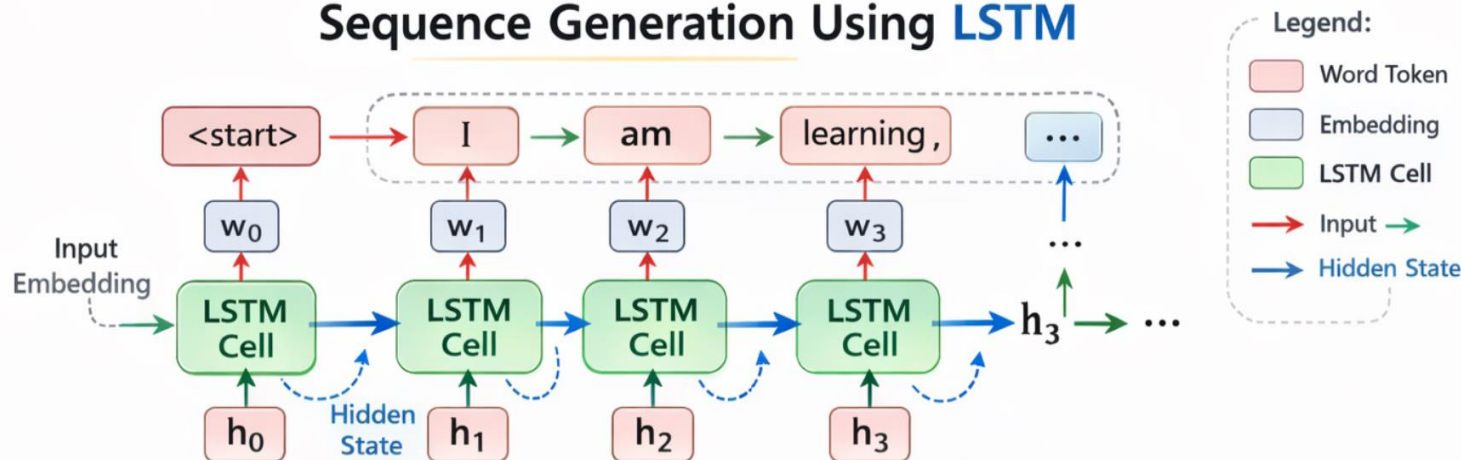
Language Models — Some milestone

- **RNN/LSTM (2013-2014), Seq2Seq Time series modelling**
 - Word2Vec understands the meaning of individual words, it cannot process long sequences of text (it is static). **Time series modelling** enable machines to read sentences from left to right.

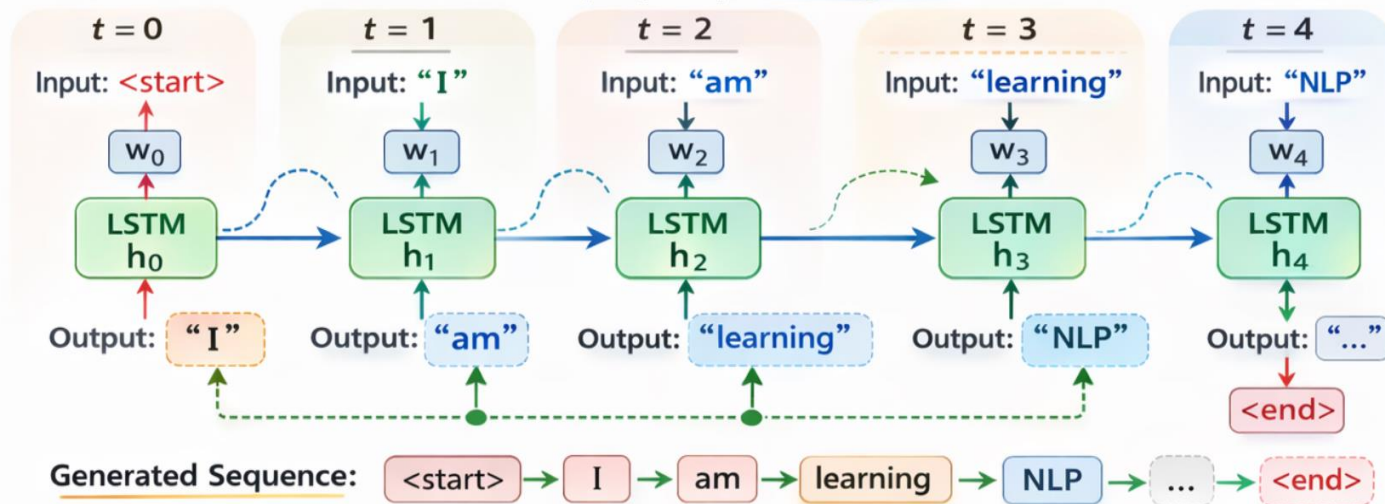


Language Models — Some milestone

Sequence Generation Using LSTM



Generation Step by Step (Autoregressive)

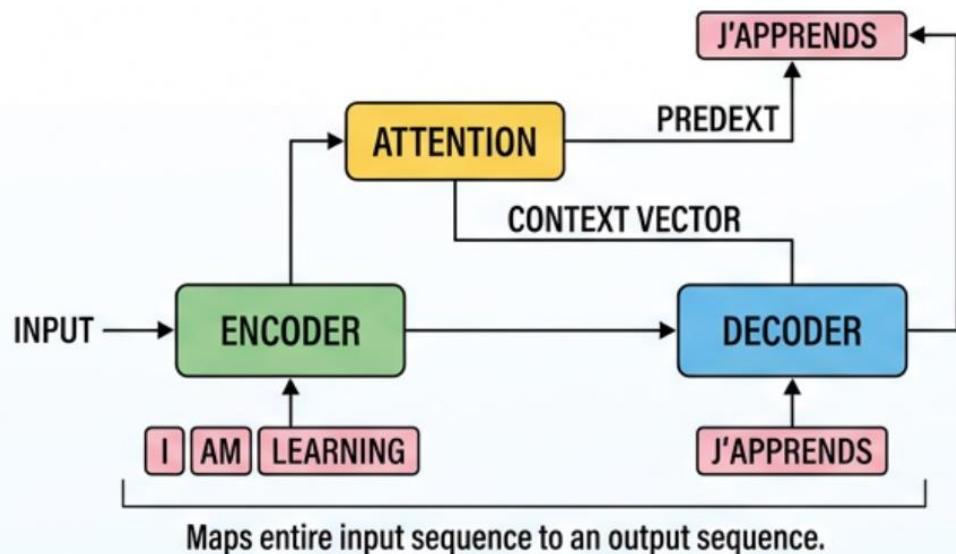


LSTM learns hidden states h_t to generate the next word at each step.

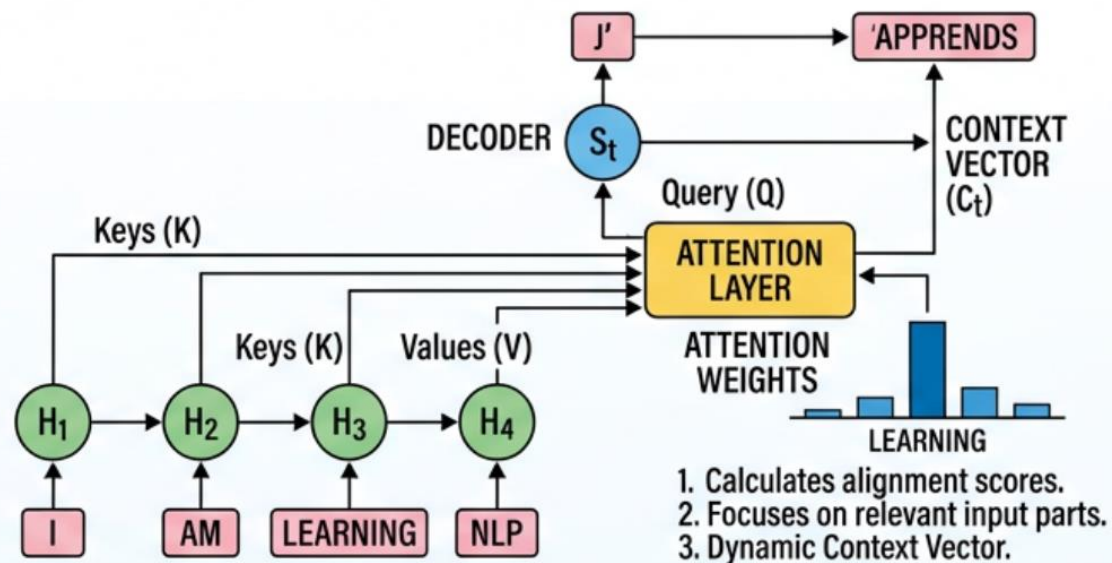
Language Models — Some milestone

- Seq2Seq Text Generation

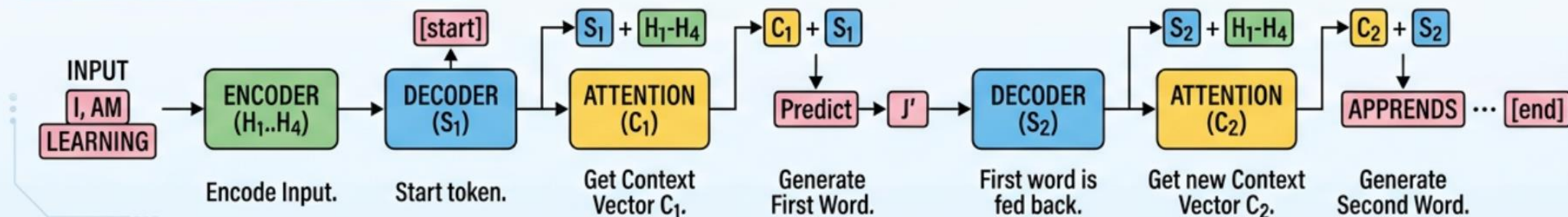
THE OVERALL ARCHITECTURE



DETAILED VIEW: ATTENTION PROCESS



STEP-BY-STEP GENERATION

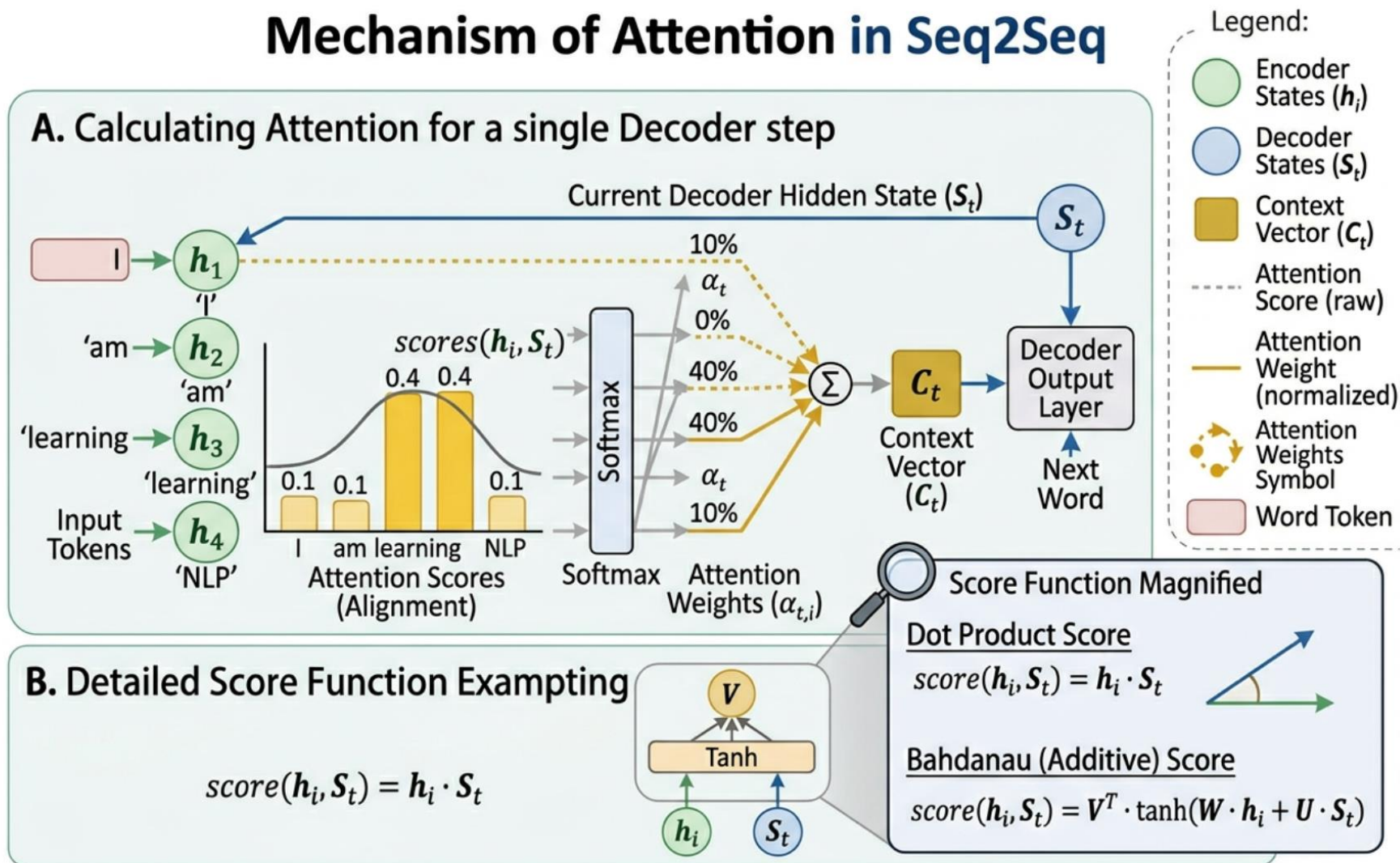


KEY

- ENCODER
- DECODER
- ATTENTION
- INPUT/OUTPUT WORD
- HIDDEN STATE
- ARROW OF FLOW

Language Models — Some milestone

Mechanism of Attention in Seq2Seq



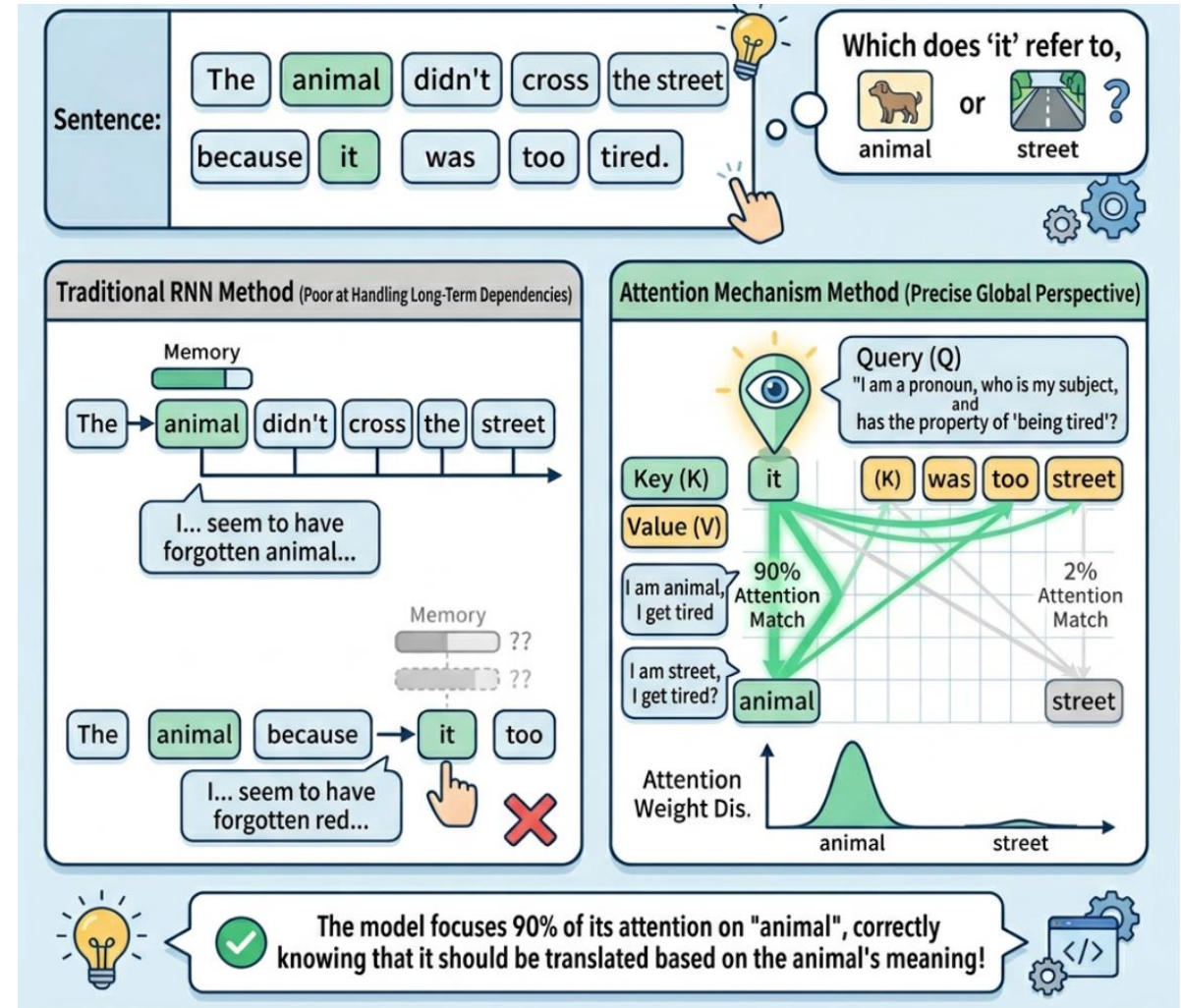
Attention dynamically weights encoder states for each decoder step, focusing on relevant input context.

Recurrent NN is not enough

- **Inability to Parallelize (Extremely Slow):**
 - It must finish processing the first word before it can move on to the second.
- **Information Bottleneck / Forgetting:**
 - The Seq2Seq architecture requires forcing an entire, highly complex long sentence to be compressed into a single, fixed-size "little notebook" (a vector

Language Models — Attention Mechanism

When generating each new word, the machine dynamically "looks back" at all the words on the table and decides which word to focus its "attention (weight)" on.



Language Models — Transformer

Introduced in 2017 in Attention Is All You Need paper(Vaswani et al.)

Attention Is All You Need

Ashish Vaswani*
Google Brain
avaswani@google.com

Noam Shazeer*
Google Brain
noam@google.com

Niki Parmar*
Google Research
nikip@google.com

Jakob Uszkoreit*
Google Research
usz@google.com

Llion Jones*
Google Research
llion@google.com

Aidan N. Gomez* †
University of Toronto
aidan@cs.toronto.edu

Łukasz Kaiser*
Google Brain
lukaszkaizer@google.com

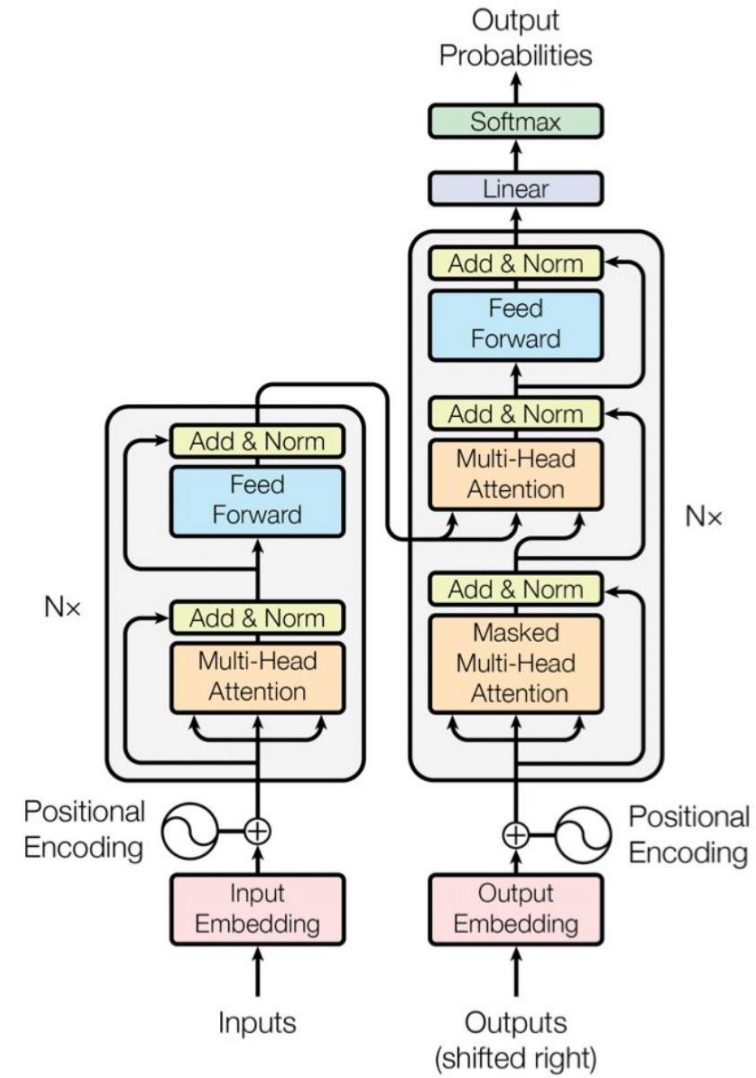
Illia Polosukhin* ‡
illia.polosukhin@gmail.com

Abstract

The dominant sequence transduction models are based on complex recurrent or convolutional neural networks that include an encoder and a decoder. The best performing models also connect the encoder and decoder through an attention mechanism. We propose a new simple network architecture, the Transformer, based solely on attention mechanisms, dispensing with recurrence and convolutions entirely. Experiments on two machine translation tasks show these models to be superior in quality while being more parallelizable and requiring significantly less time to train. Our model achieves 28.4 BLEU on the WMT 2014 English-to-German translation task, improving over the existing best results, including ensembles, by over 2 BLEU. On the WMT 2014 English-to-French translation task, our model establishes a new single-model state-of-the-art BLEU score of 41.8 after training for 3.5 days on eight GPUs, a small fraction of the training costs of the best models from the literature. We show that the Transformer generalizes well to other tasks by applying it successfully to English constituency parsing both with

Transformer

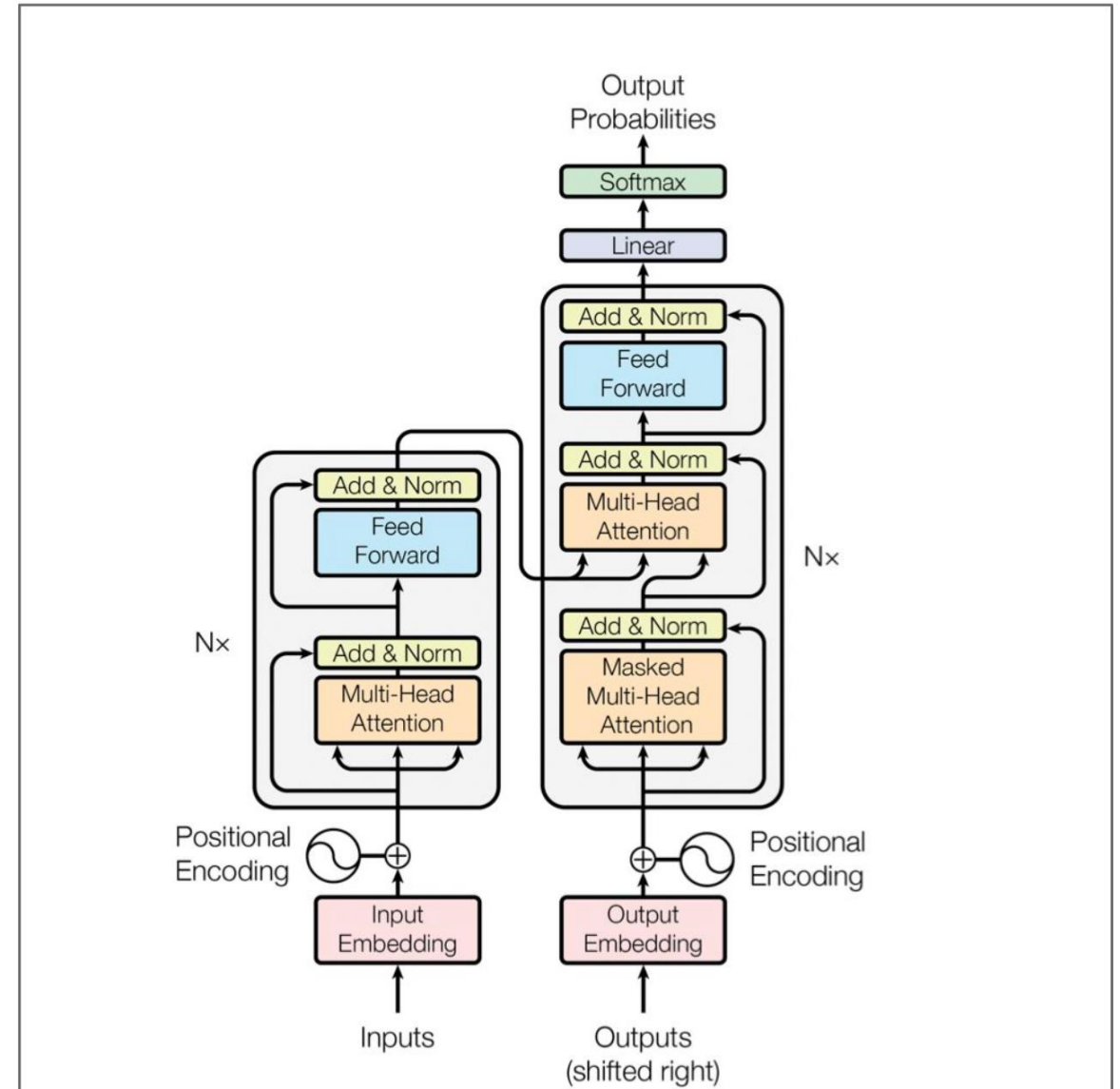
Transformer is neural network architecture that can process sequential data such as texts, audio, video. Transformer has also shown to work well in data that are not necessarily sequential such as images.



Vaswani et al., "Attention is all you need", 2017

Recap — Transformer

- Tokenization
- Input Embeddings
- Position Encodings
- Query, Key, & Value
- Attention
- Self Attention
- Multi-Head Attention
- Feed Forward
- Add & Norm
- Encoders
- Masked Attention
- Encoder Decoder Attention
- Linear • Softmax
- Decoders
- Encoder-Decoder Models



Vaswani et al., "Attention is all you need", 2017

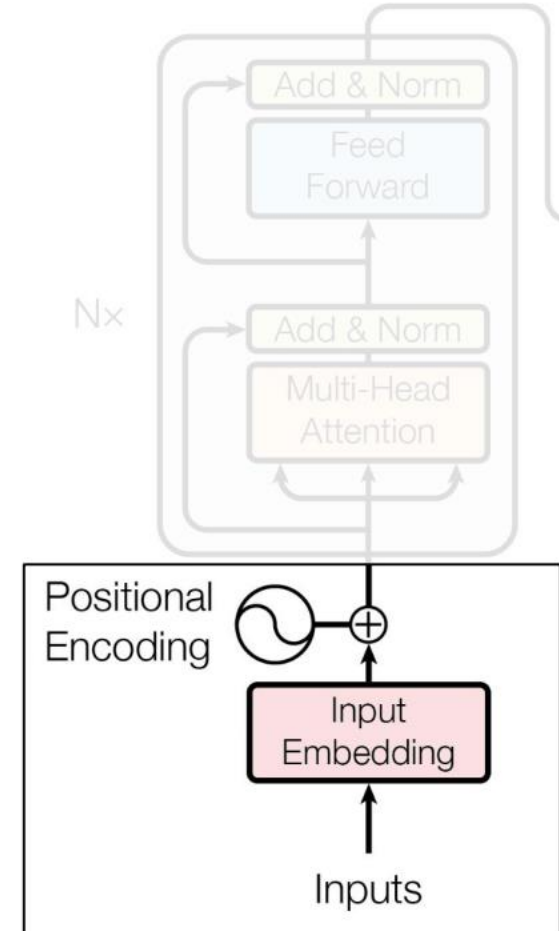
Transformer — Tokenization

Tokenizer split sentences into individual words.

```
texts[i] = "the cat sat on the mat."
```

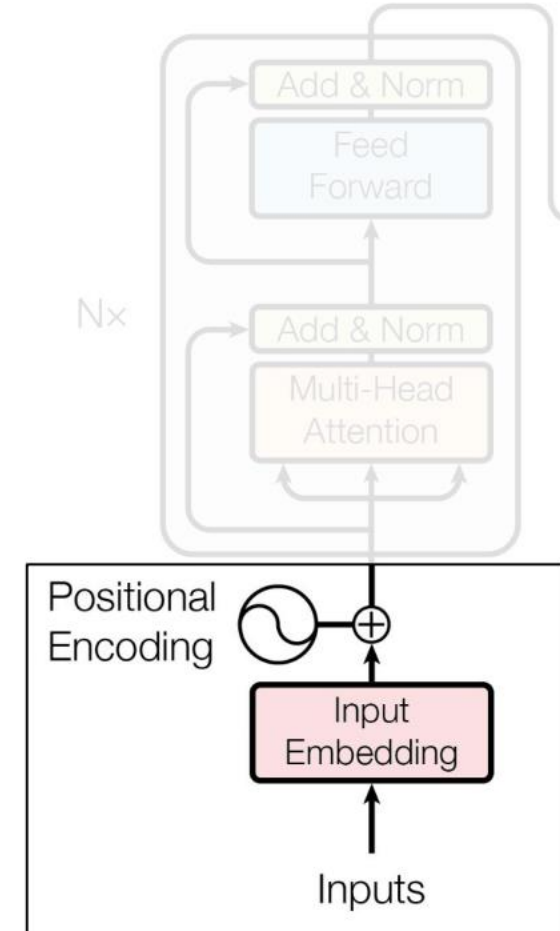
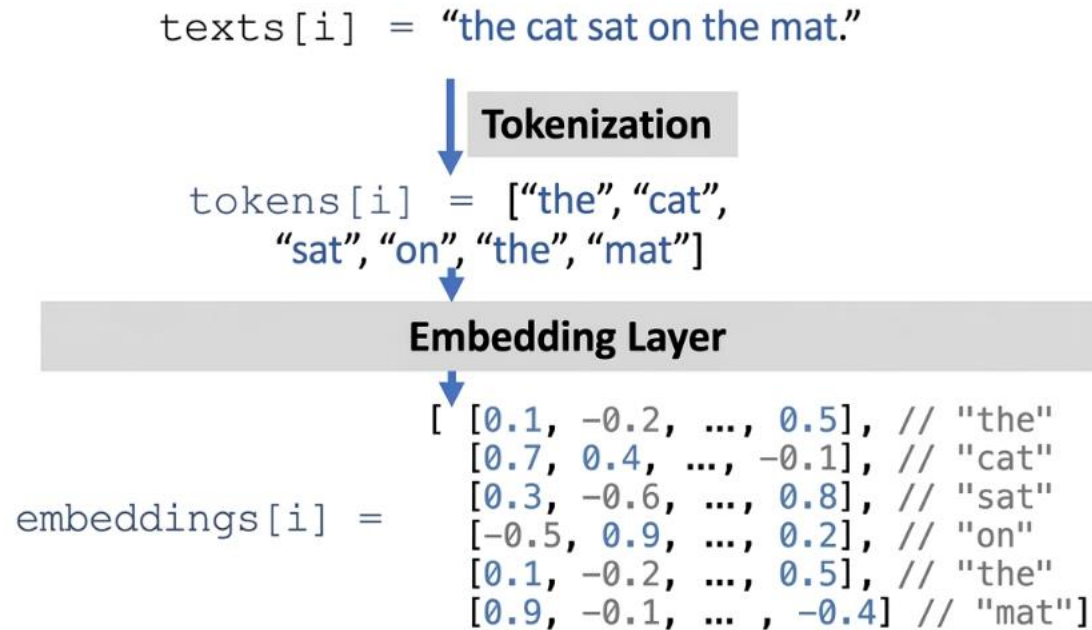
Tokenization

```
tokens[i] = ["the", "cat",  
            "sat", "on", "the", "mat"]
```



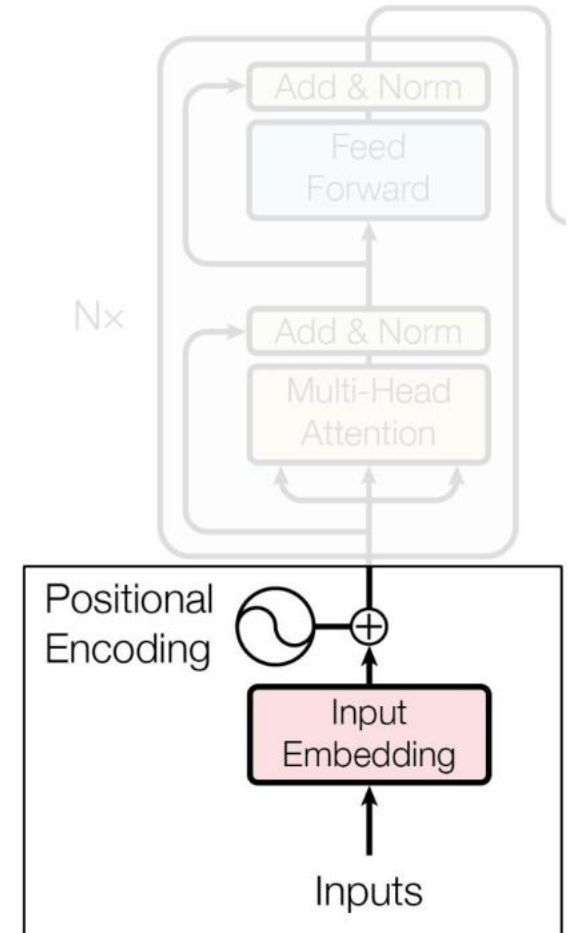
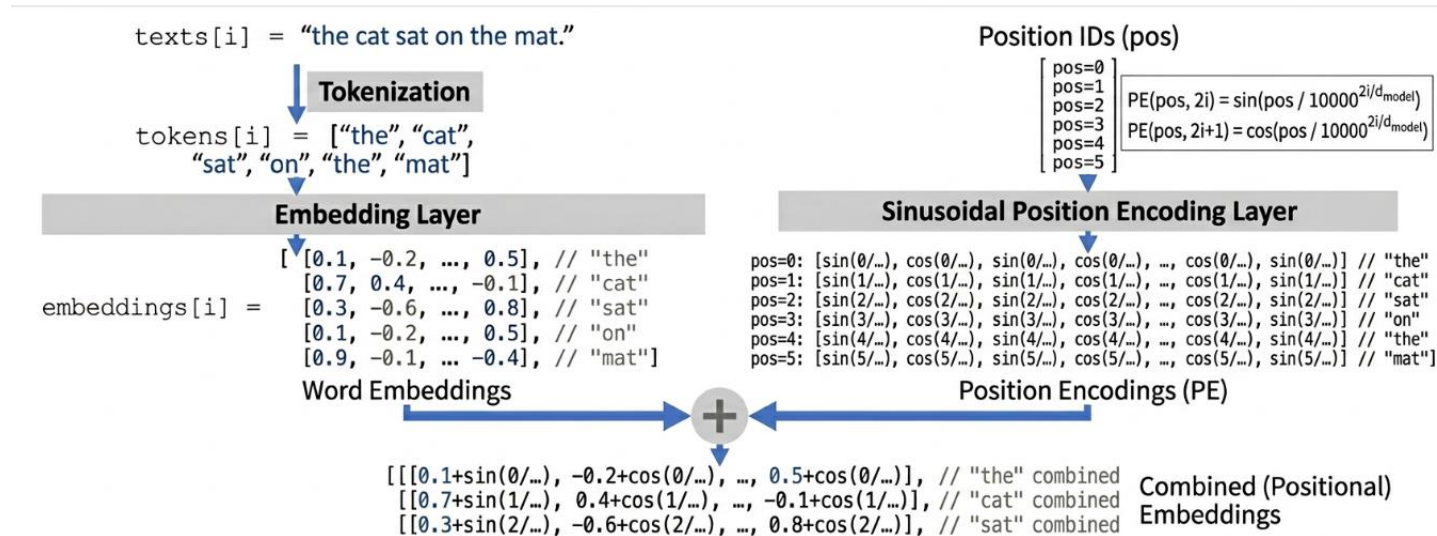
Transformer — Embedding

The words will project to embeddings via embedding layer.



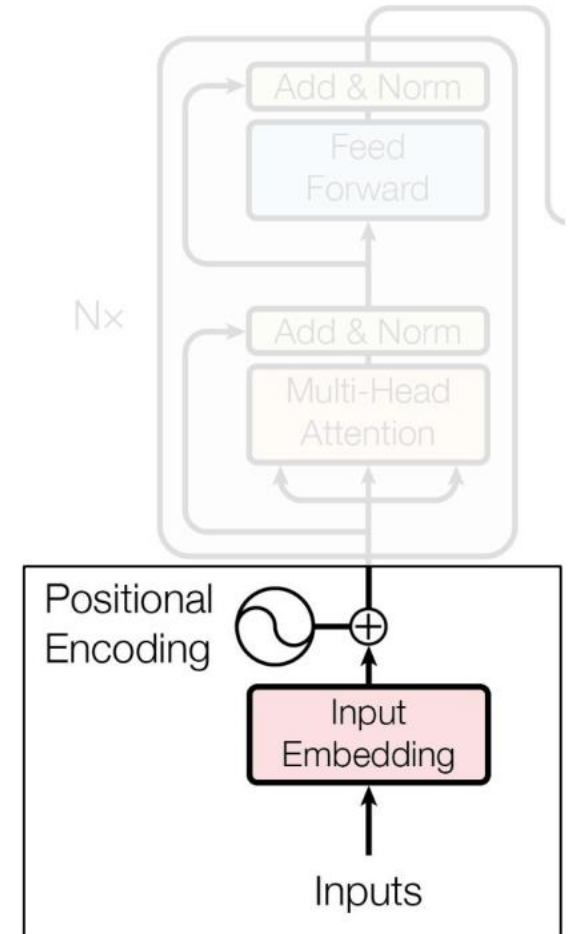
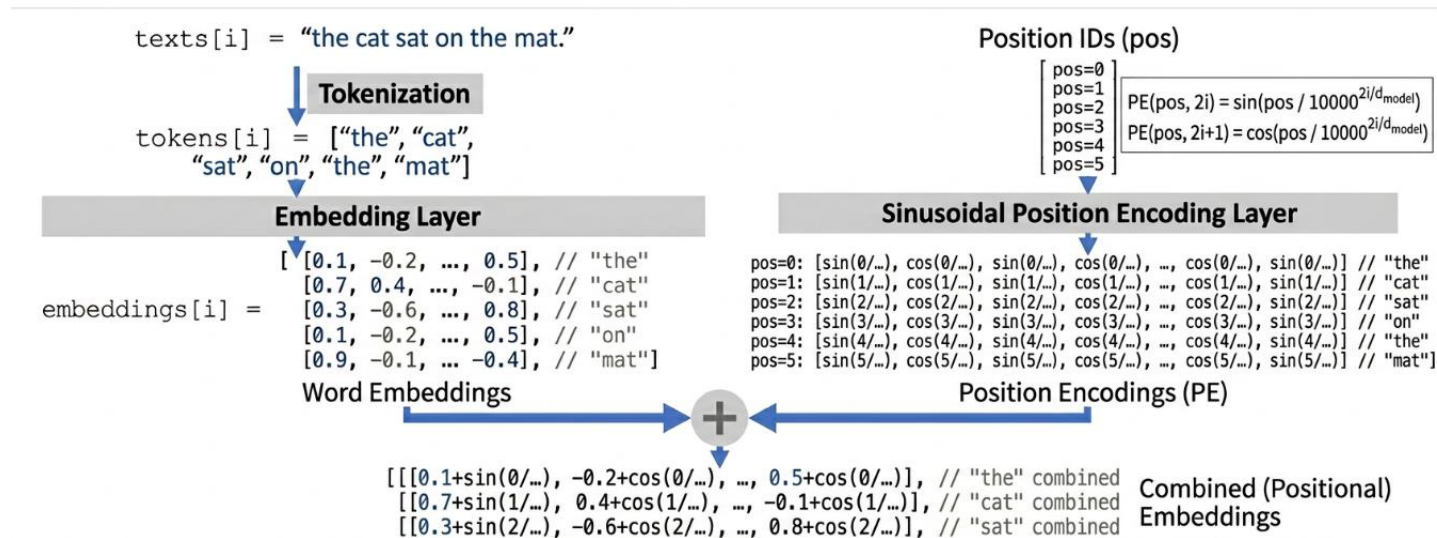
Transformer — Position Encode

The position information are embedded



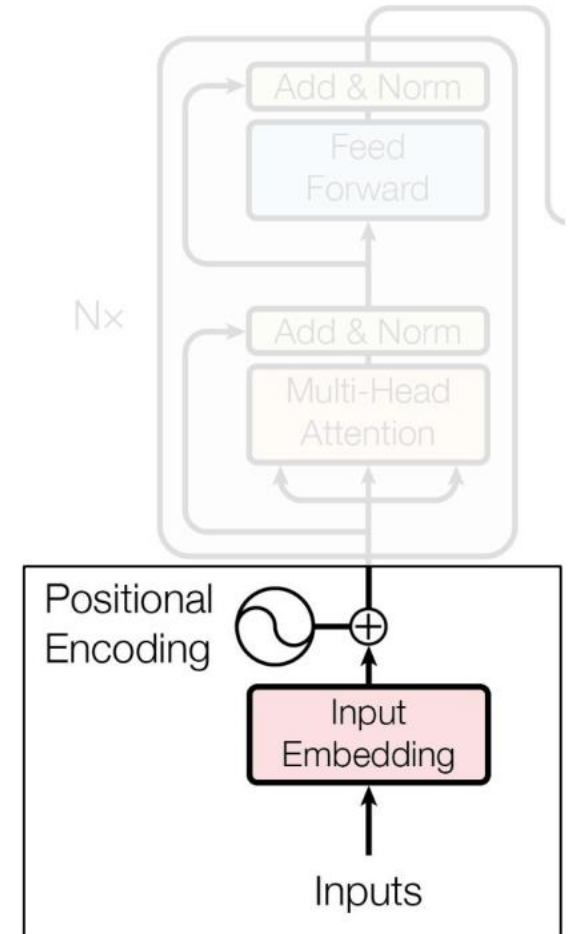
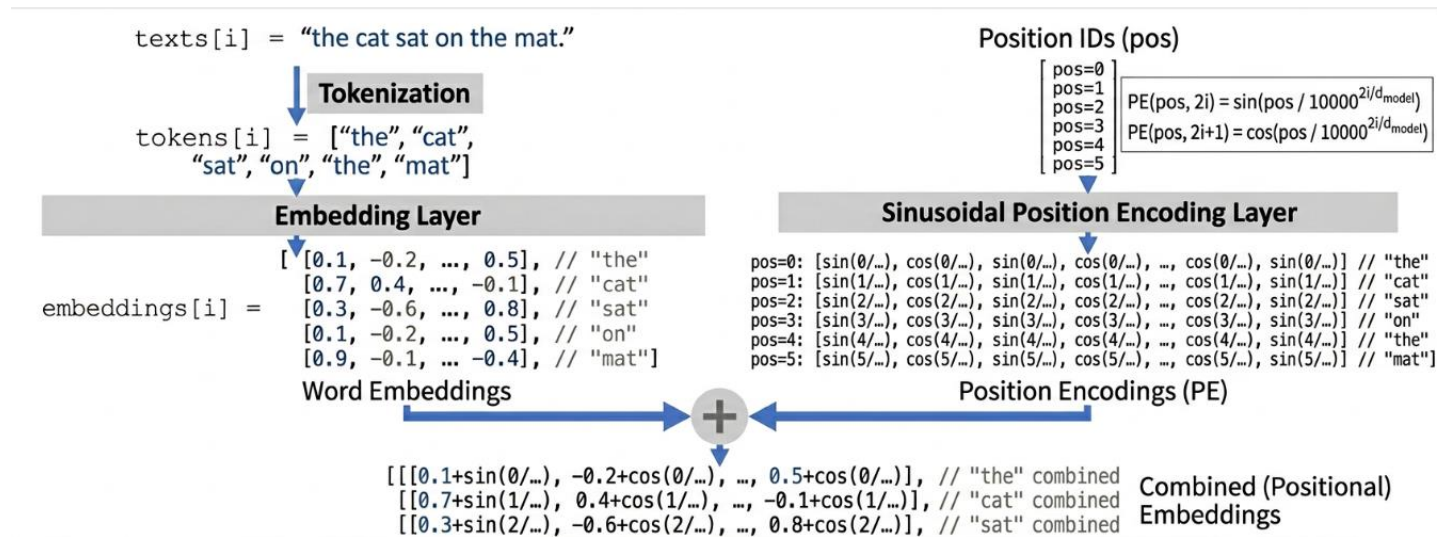
Transformer — Position Encode

Handle variable sentence lengths: Sinusoidal encodings don't need retraining for different lengths; they're calculated for any position. This enables the model to handle varying input lengths effectively.



Transformer — Position Encode

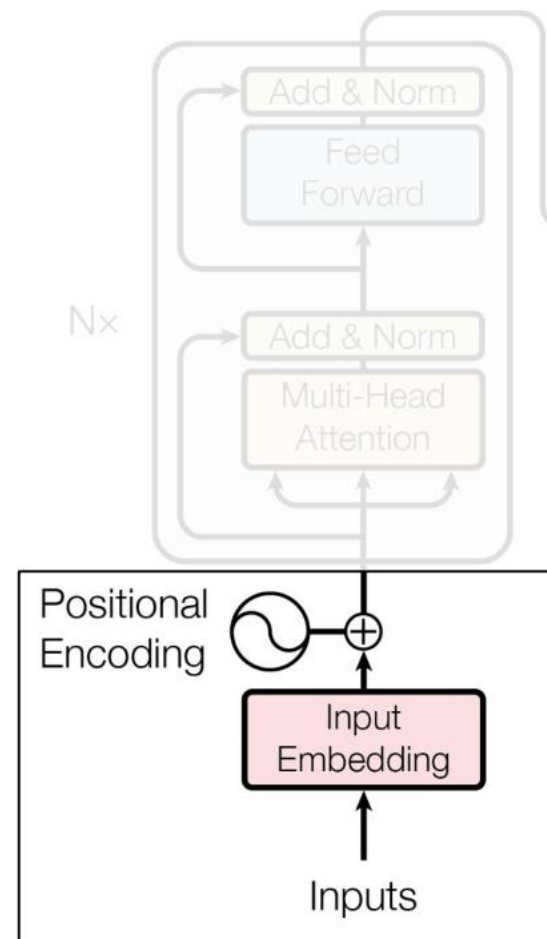
Simple computation and generalization: Generating these encodings involves straightforward math, avoiding extra parameters. Research suggests this helps the model generalize to unseen positional information.



Transformer — Rotary Position Embedding

RoFORMER: ENHANCED TRANSFORMER WITH ROTARY POSITION EMBEDDING

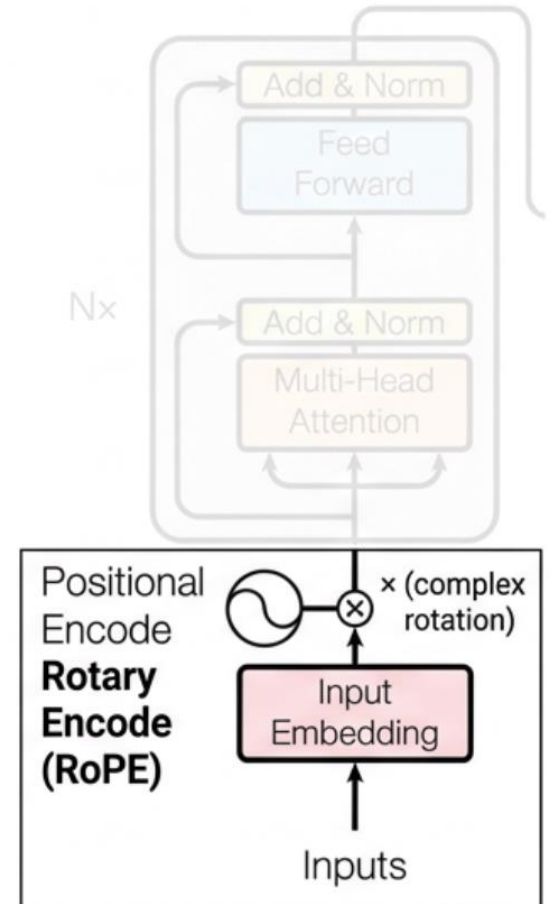
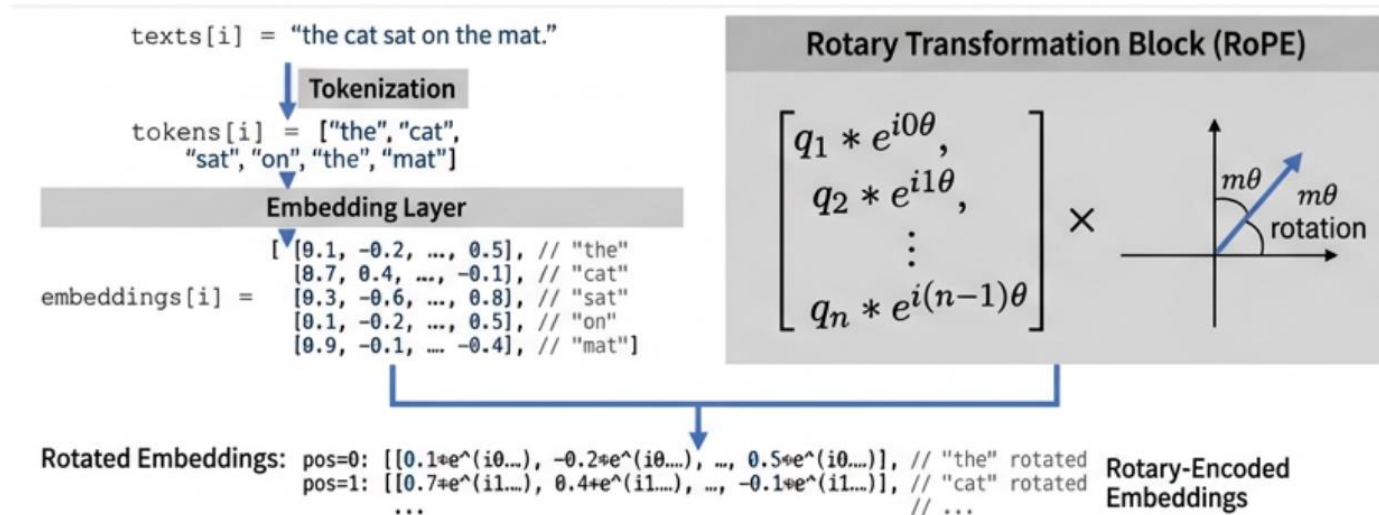
$$f_{\{q,k\}}(\mathbf{x}_m, m) = \begin{pmatrix} \cos m\theta & -\sin m\theta \\ \sin m\theta & \cos m\theta \end{pmatrix} \begin{pmatrix} W_{\{q,k\}}^{(11)} & W_{\{q,k\}}^{(12)} \\ W_{\{q,k\}}^{(21)} & W_{\{q,k\}}^{(22)} \end{pmatrix} \begin{pmatrix} x_m^{(1)} \\ x_m^{(2)} \end{pmatrix}$$



Transformer — Rotary Position Embedding

RoPE encodes relative position by rotating word embeddings.

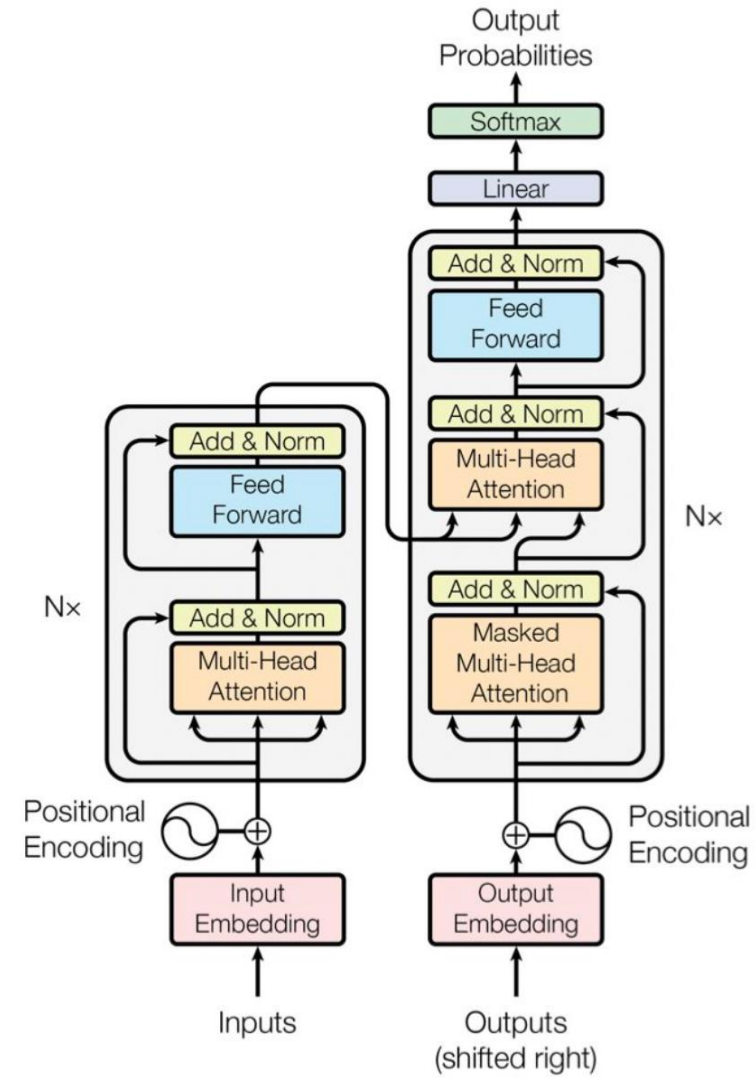
Key Benefits: Direct computation, parameter-free, and strong generalization to varying and unseen text lengths.



Transformer

It's principal layer is called Attention. It also contain other basic layers such as fully-connected layers, layer normalization, embedding layer, and positional encoding layer for maintaining the order of tokens.

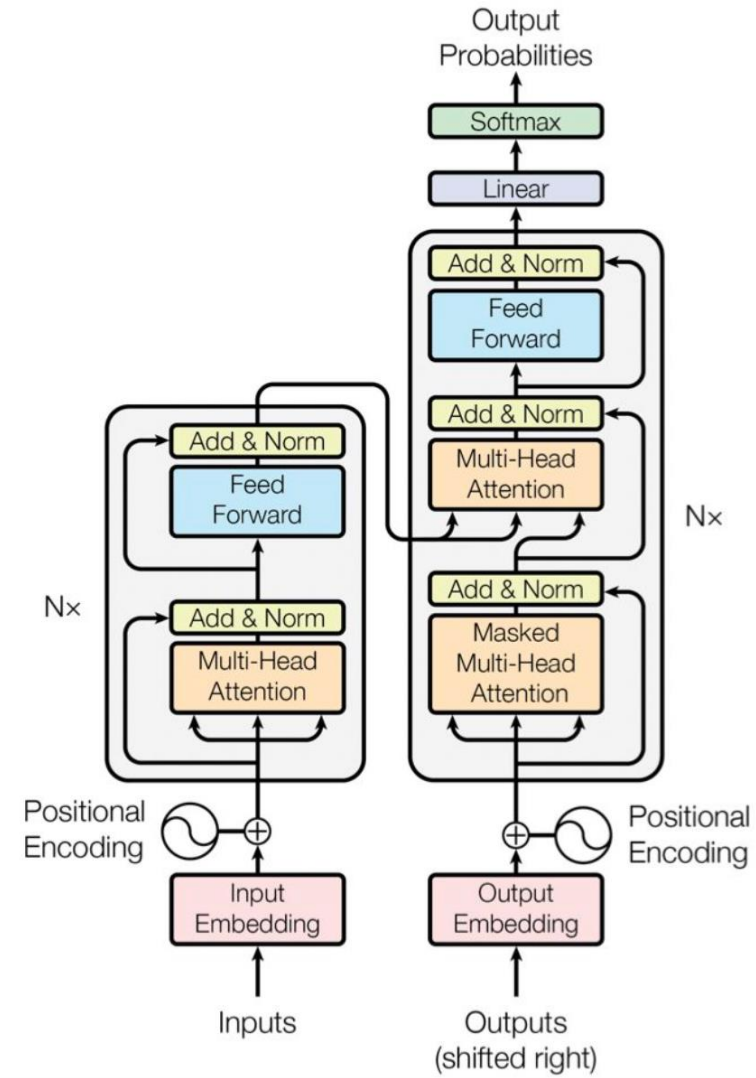
Original Transformer was made of two parts: **Encoder** and Decoder



Vaswani et al., "Attention is all you need", 2017

Transformer

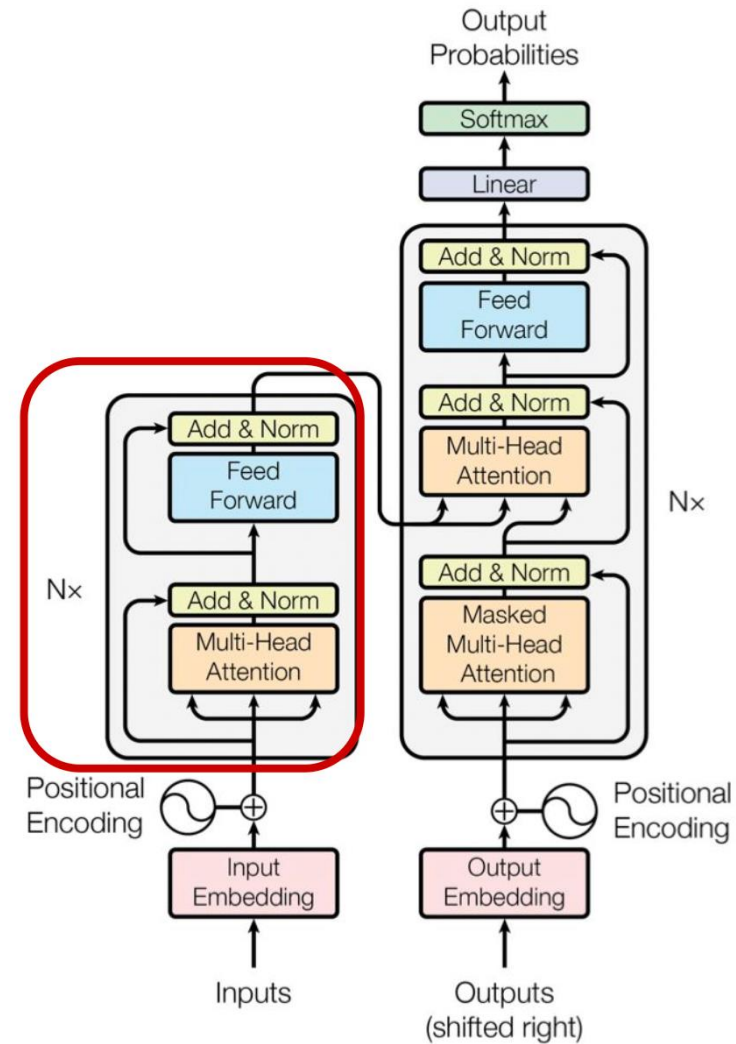
Transformer is an autoregressive architecture. It generate one token at time until the whole output sequence is generated. The current token to be generated is conditioned on previous generated tokens



Vaswani et al., "Attention is all you need", 2017

Encoder

Encoder is one of the main building block of Transformer that transform the input sequence into compressed representation. It's made of 6 identical blocks, each block having 3 main layers: multi-head attention, layer norm, MLPs(feedforward).



Vaswani et al., "Attention is all you need", 2017

Transformer — Encoder – Self-Attention

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

Query

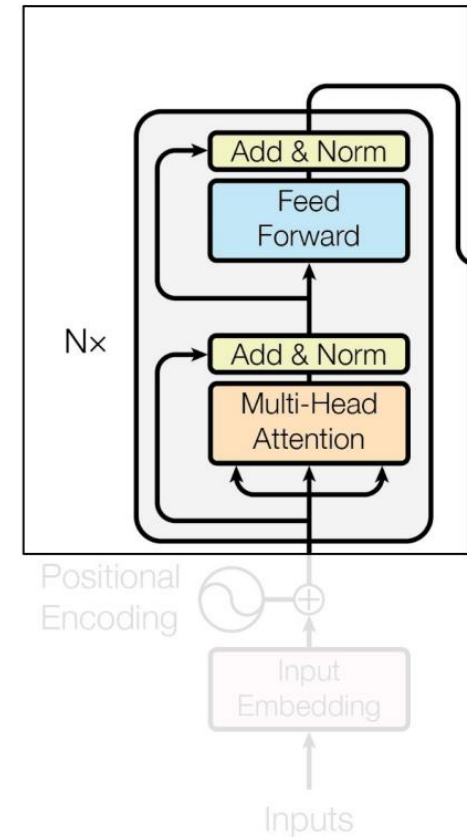
1. Search for info

Key

1. Interacts directly with Queries
2. Distinguishes one object from another
3. Identify which object is the most relevant and by how much

Value

1. Actual details of the object
2. More fine grained



Transformer — Encoder – Self-Attention

Let's look at the sentence: "**Apple company released a new phone.**"

In a Transformer model, **every word acts as a Q, K, and V all at the same time**. Let's focus on how the word "**Apple**" processes its meaning.

1. Q (Query): "Apple" sends out a search request (Query) to the whole sentence: *"I need to update my meaning! Can anyone give me clues? Am I a tasty fruit or a business?"*

2. K (Key): Every word in the sentence (including "Apple" itself) holds up its ID tag (Key).

- "Company" holds a Key tagged *"Organization"*.
- "Phone" holds a Key tagged *"Electronic Device"*.

Transformer — Encoder – Self-Attention

3. Scoring ($Q \cdot K^T$): "Apple" goes around shaking hands (calculating dot products) with every word's Key to see how well they match.

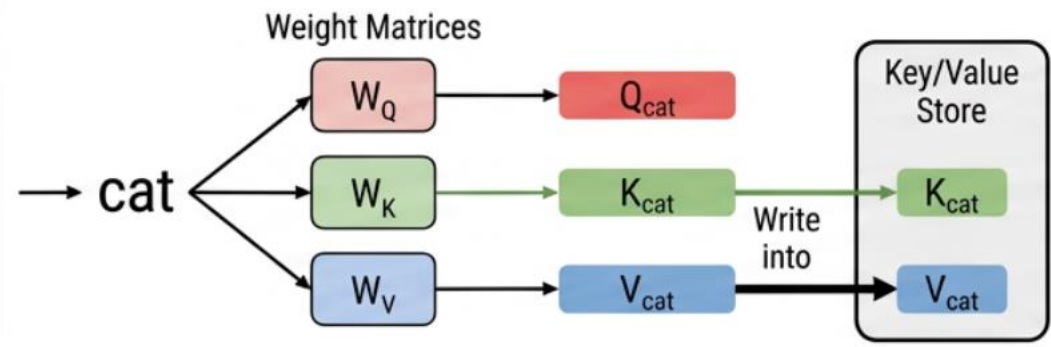
- "Apple" Q shakes hands with "Company" K → **Huge match! (90 points)**
- "Apple" Q shakes hands with "released" K → **Okay match. (50 points)**
- "Apple" Q shakes hands with "a" K → **Terrible match. (1 point)**

4. V (Value): This is the actual, core meaning (content) hidden inside each word.

5. Final Aggregation: The scores are turned into percentages using Softmax (e.g., "Company" gets 80%, "Phone" gets 15%). "Apple" then **absorbs the actual meanings (Values)** of the other words based on these exact percentages.

Transformer — Encoder – Self-Attention

STEP 1: From Word to Role (Projections)



Q (● Search query): Search for place details
 K (● Identity Tag): Object, noun
 V (● Actual Content): Feline details

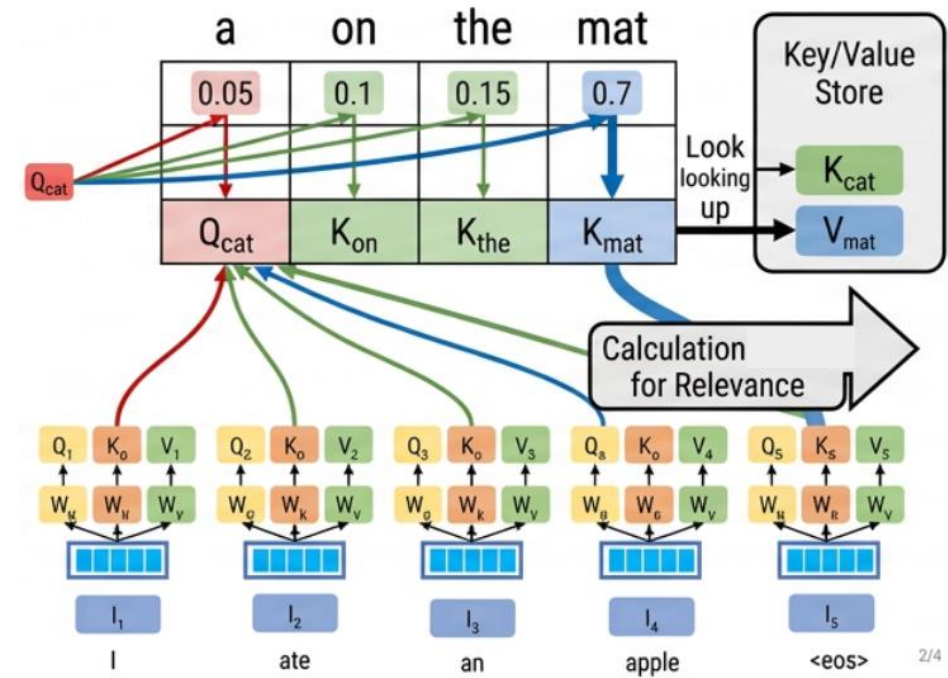


Generic object isa generic object

1/4

1/4

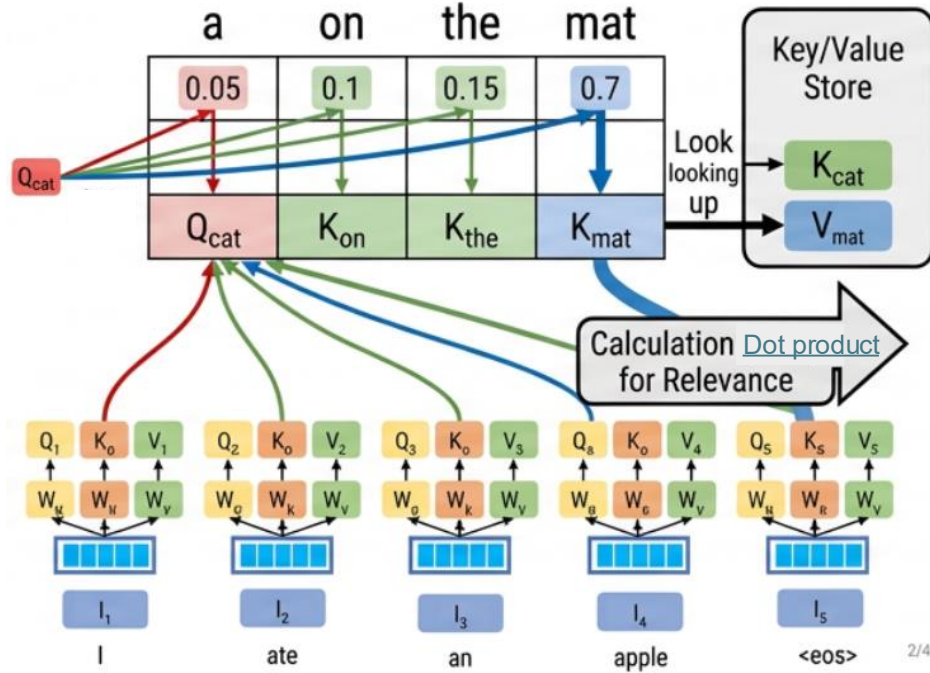
STEP 2: Finding Relevance (Scores)



2/4

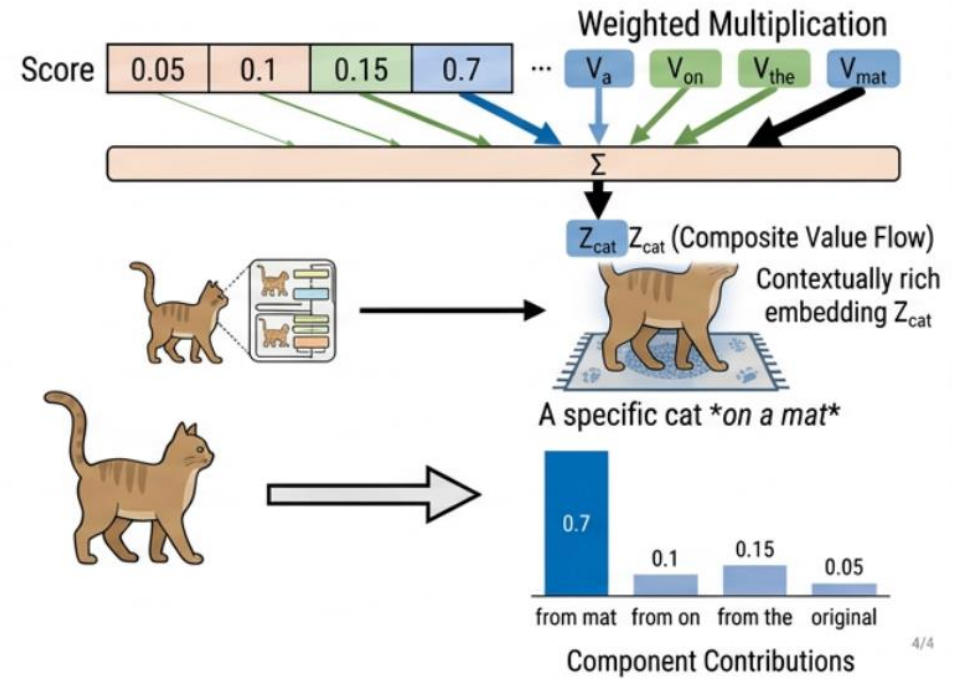
Transformer — Encoder – Self-Attention

STEP 2: Finding Relevance (Scores)



2/4

STEP 3: Information Flow (Weighted Sum)



4/4

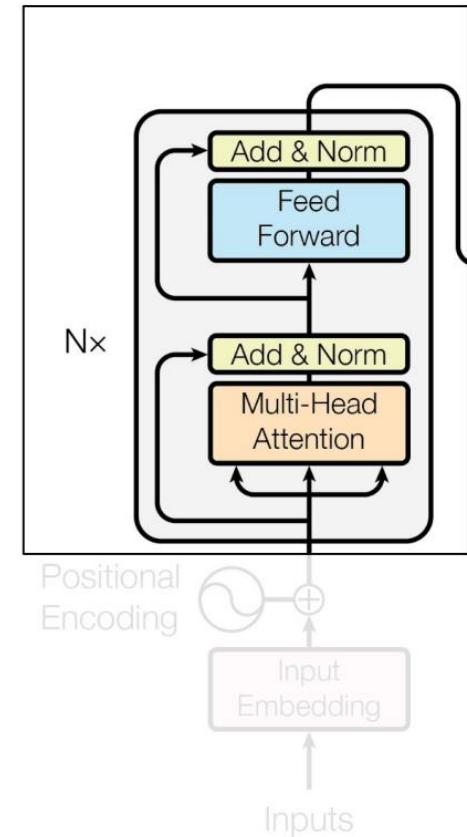
Transformer — Encoder – Self-Attention

Let's calculate Self-Attention manually.

Input: $X = [\text{"Thinking"}, \text{"Machines"}]$. Dimensions: $d_{model} = 4, d_k = 3$.

$$X = \begin{bmatrix} 1 & 0 & 1 & 0 \\ 0 & 1 & 0 & 1 \end{bmatrix}$$

(Row 1: Thinking, Row 2: Machines)



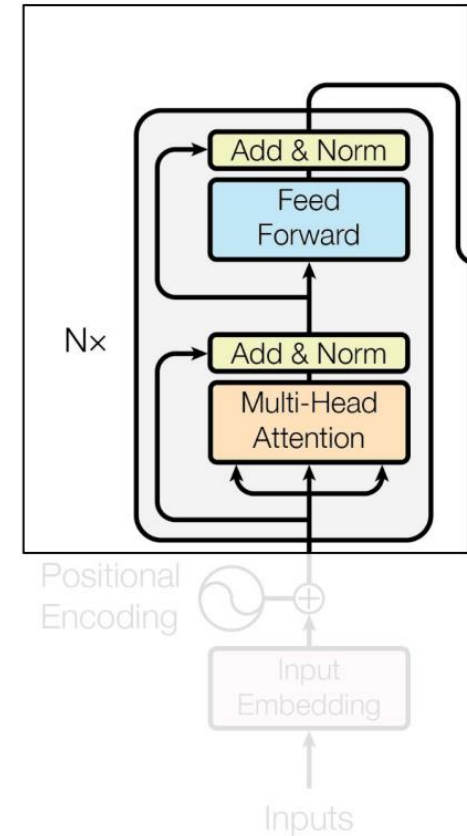
Transformer — Encoder – Self-Attention

We multiply Input X by weight matrices W^Q, W^K, W^V (learned parameters).

$$Q = X \cdot W^Q = \begin{bmatrix} 1 & 0 & 2 \\ 0 & 1 & 1 \end{bmatrix} \quad K = X \cdot W^K = \begin{bmatrix} 2 & 0 & 1 \\ 0 & 2 & 1 \end{bmatrix}$$

$$V = X \cdot W^V = \begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \end{bmatrix}$$

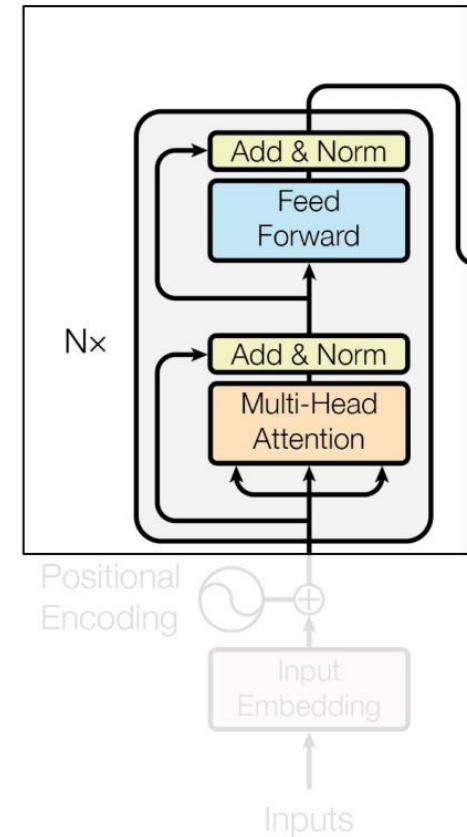
Note: Q row 1 is the Query for "Thinking". K row 2 is the Key for "Machines".



Transformer — Encoder – Self-Attention

We want to know how much "Thinking" relates to "Machines".

$$Q \cdot K^T = \begin{bmatrix} 1 & 0 & 2 \\ 0 & 1 & 1 \end{bmatrix} \cdot \begin{bmatrix} 2 & 0 \\ 0 & 2 \\ 1 & 1 \end{bmatrix}$$



Transformer — Encoder – Self-Attention

We want to know how much "Thinking" relates to "Machines".

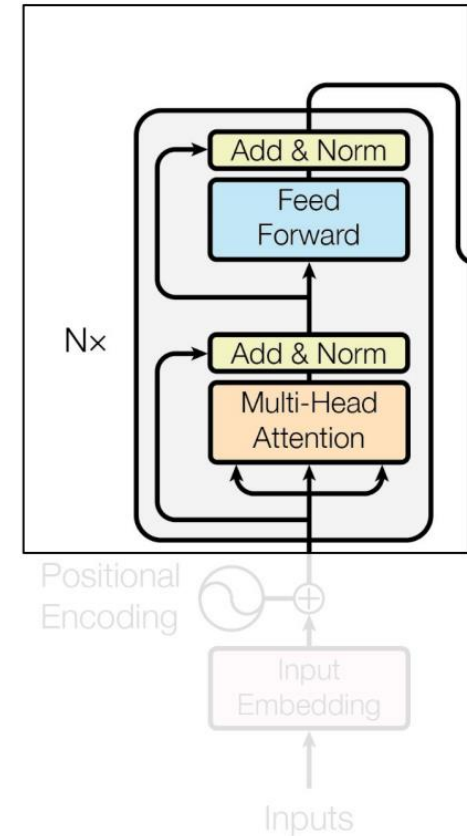
$$Q \cdot K^T = \begin{bmatrix} 1 & 0 & 2 \\ 0 & 1 & 1 \end{bmatrix} \cdot \begin{bmatrix} 2 & 0 \\ 0 & 2 \\ 1 & 1 \end{bmatrix}$$

Row 1 (Thinking) · Col 1 (Thinking): $(1 \times 2) + (0 \times 0) + (2 \times 1) = 4$

Score(Think, Think)

Row 1 (Thinking) · Col 2 (Machines): $(1 \times 0) + (0 \times 2) + (2 \times 1) = 2$

Score(Think, Machine)



Transformer — Encoder – Self-Attention

Row 1 (Thinking) · Col 1 (Thinking): $(1 \times 2) + (0 \times 0) + (2 \times 1) = 4$

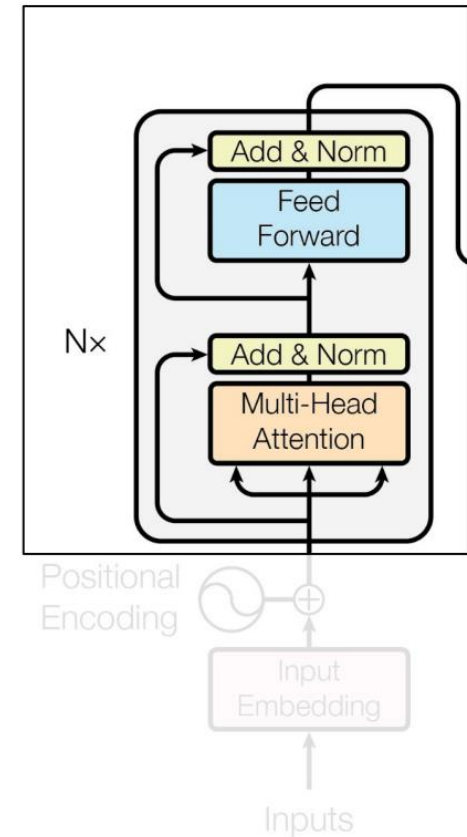
Score(Think, Think)

Row 1 (Thinking) · Col 2 (Machines): $(1 \times 0) + (0 \times 2) + (2 \times 1) = 2$

Score(Think, Machine)

After computing all dot products:

$$\text{Scores} = \begin{bmatrix} 4 & 2 \\ 1 & 3 \end{bmatrix}$$



Transformer — Encoder – Self-Attention

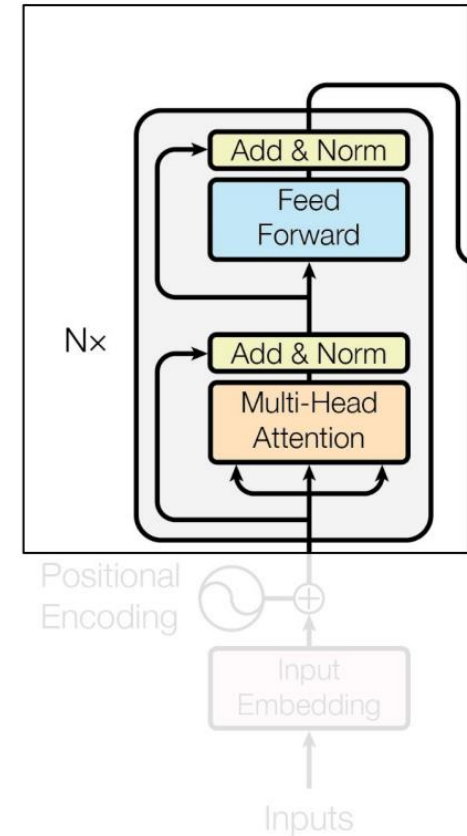
After computing all dot products:

$$\text{Scores} = \begin{bmatrix} 4 & 2 \\ 1 & 3 \end{bmatrix}$$

Step 3: Scaling ($\sqrt{d_k}$). Let's say $\sqrt{d_k} = 2$.

$$\text{Scaled} = \begin{bmatrix} 4/2 & 2/2 \\ 1/2 & 3/2 \end{bmatrix} = \begin{bmatrix} 2.0 & 1.0 \\ 0.5 & 1.5 \end{bmatrix}$$

Why scale? To prevent gradients from vanishing in Softmax.



Transformer — Encoder – Self-Attention

Apply Softmax row-wise (probability distribution).

Row 1: $[2.0, 1.0] \xrightarrow{\text{softmax}} \left[\frac{e^2}{e^2+e^1}, \frac{e^1}{e^2+e^1} \right] \approx [0.73, 0.27]$

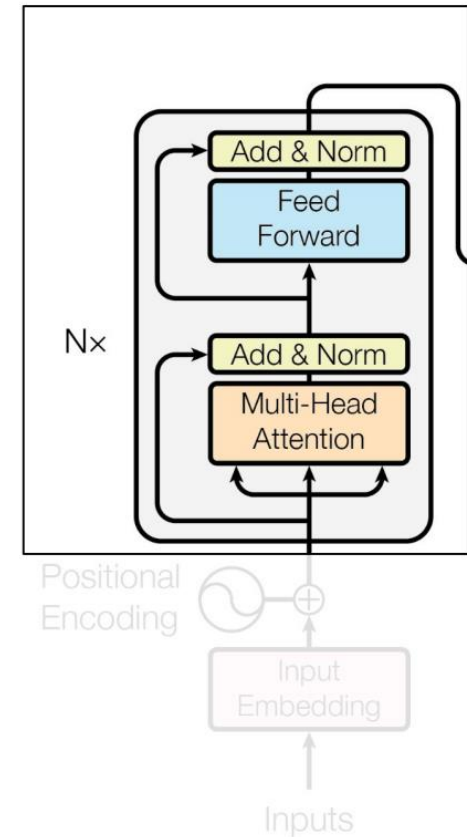
Row 2: $[0.5, 1.5] \xrightarrow{\text{softmax}} \approx [0.27, 0.73]$

$$\text{Weights} = \begin{bmatrix} 0.73 & 0.27 \\ 0.27 & 0.73 \end{bmatrix}$$

Multiply by Values (V)

$$Z = \text{Weights} \cdot V$$

$$\begin{bmatrix} 0.73 & 0.27 \\ 0.27 & 0.73 \end{bmatrix} \cdot \begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \end{bmatrix}$$



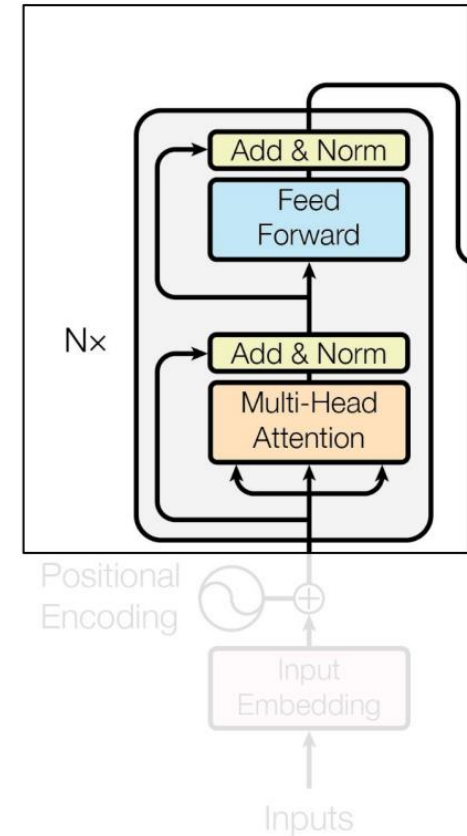
Transformer — Encoder – Self-Attention

$$Z = \text{Weights} \cdot V$$

$$\begin{bmatrix} 0.73 & 0.27 \\ 0.27 & 0.73 \end{bmatrix} \cdot \begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \end{bmatrix}$$

Result for "Thinking" (z_1): $0.73 \cdot [1, 2, 3] + 0.27 \cdot [4, 5, 6]$
 $= [0.73, 1.46, 2.19] + [1.08, 1.35, 1.62]$
 $= [1.81, 2.81, 3.81]$

Intuition: The vector for "Thinking" has absorbed 27% of the "Machines" vector!

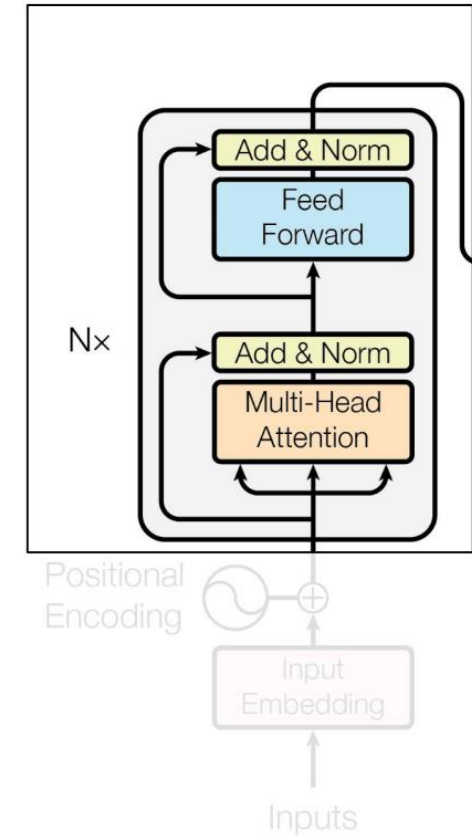
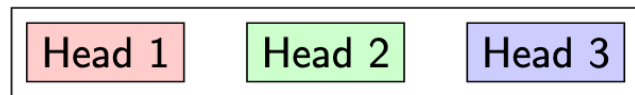


Transformer — Encoder – Multi Head Attention

Why one head is not enough? "The animal didn't cross the street because **it** was too tired."

- **Head 1 (Grammar):** "it" links to "cross" (Subject-Verb).
- **Head 2 (Meaning):** "it" links to "animal" (Reference).

Concat + Linear

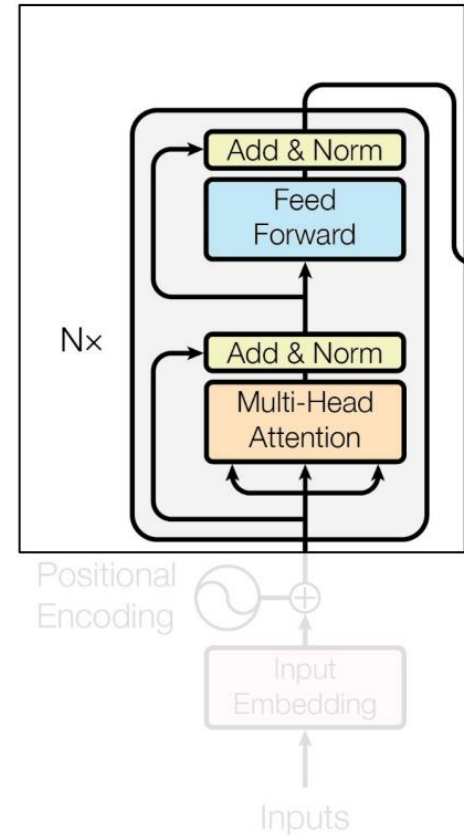
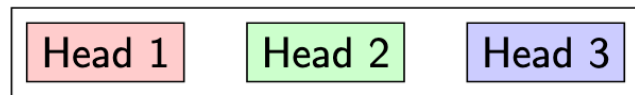


Transformer — Encoder – Multi Head Attention

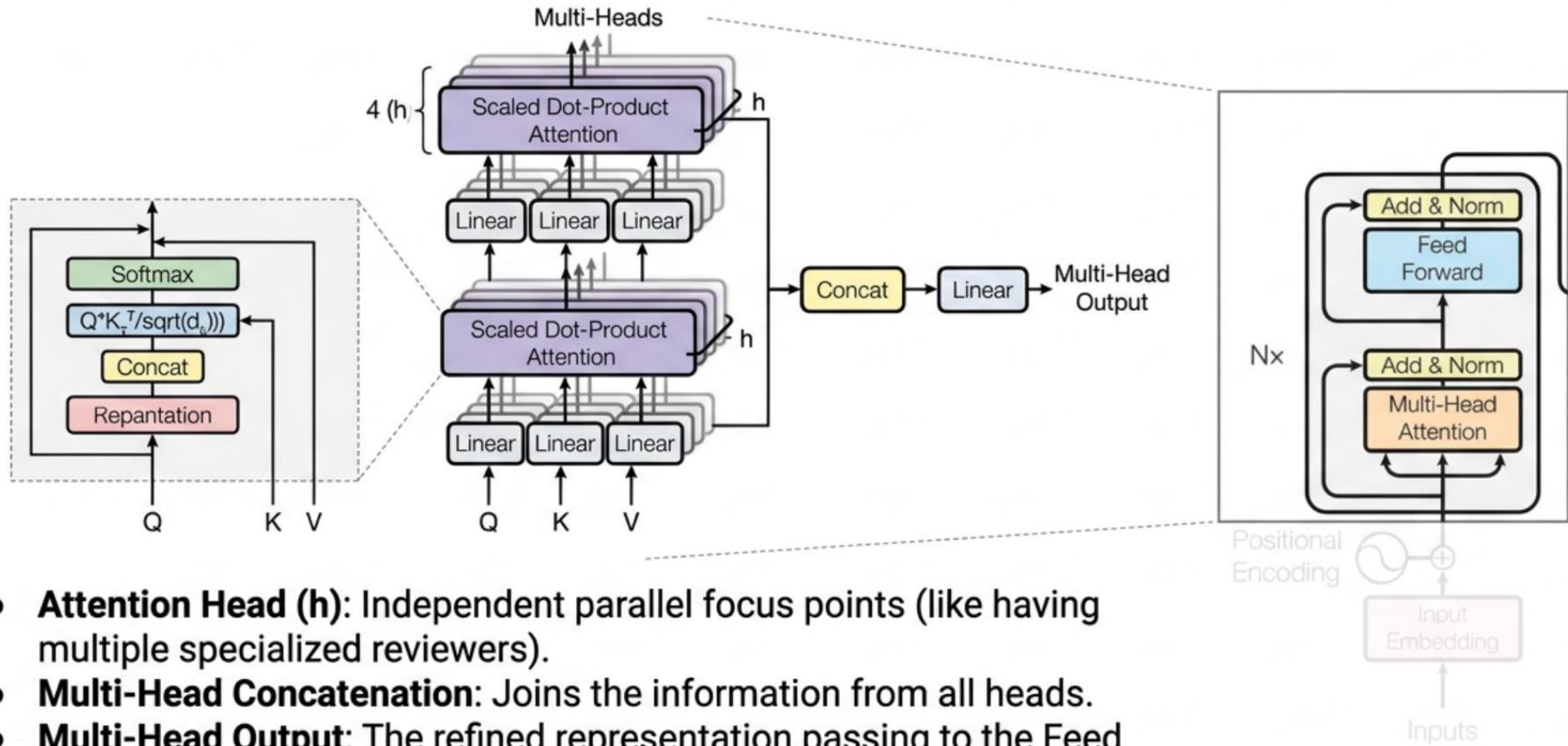
Why one head is not enough? "The animal didn't cross the street because **it** was too tired."

- **Head 1 (Grammar):** "it" links to "cross" (Subject-Verb).
- **Head 2 (Meaning):** "it" links to "animal" (Reference).

Concat + Linear

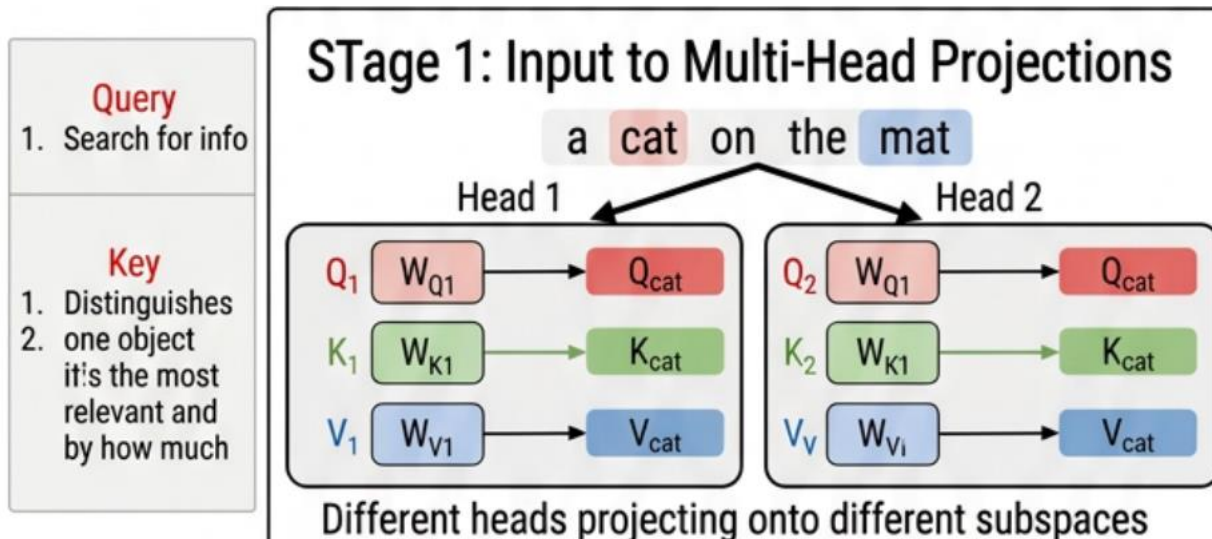


Transformer — Encoder – Multi Head Attention

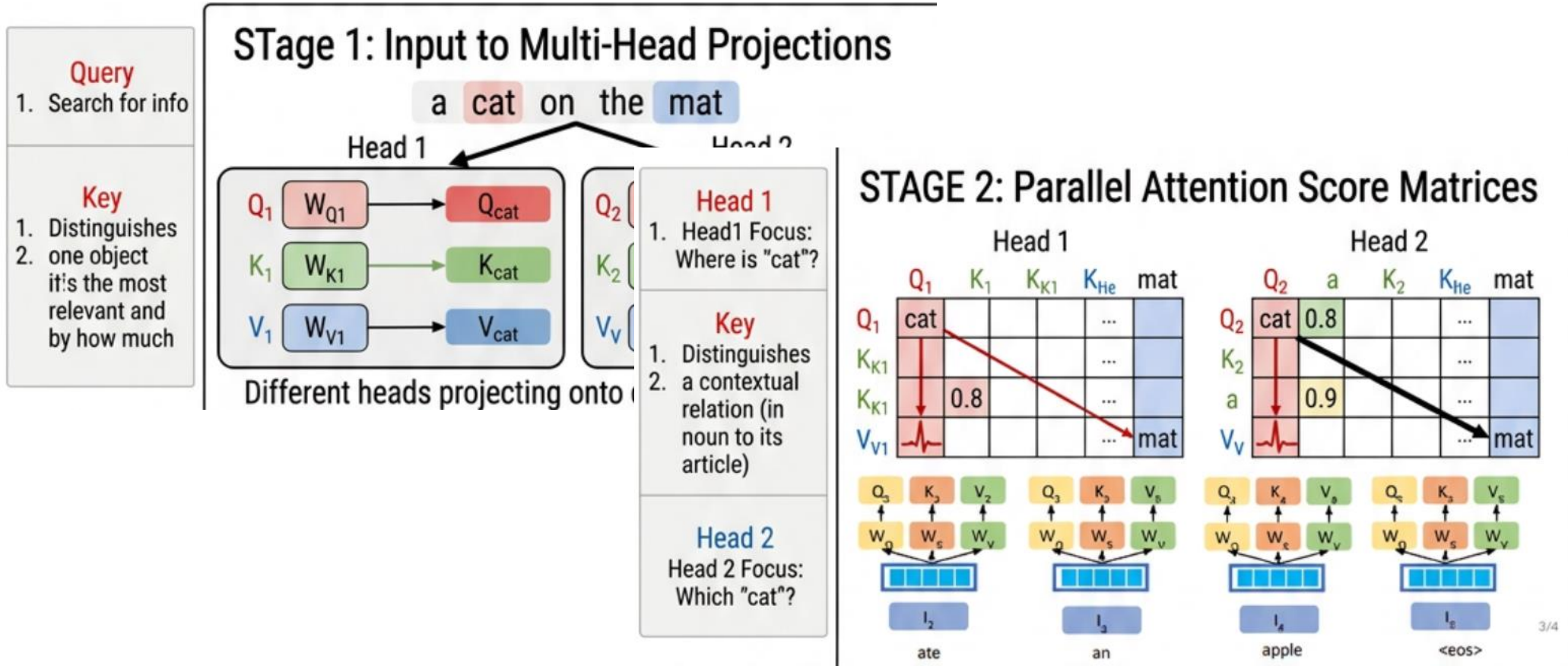


- **Attention Head (h):** Independent parallel focus points (like having multiple specialized reviewers).
- **Multi-Head Concatenation:** Joins the information from all heads.
- **Multi-Head Output:** The refined representation passing to the Feed Forward network.

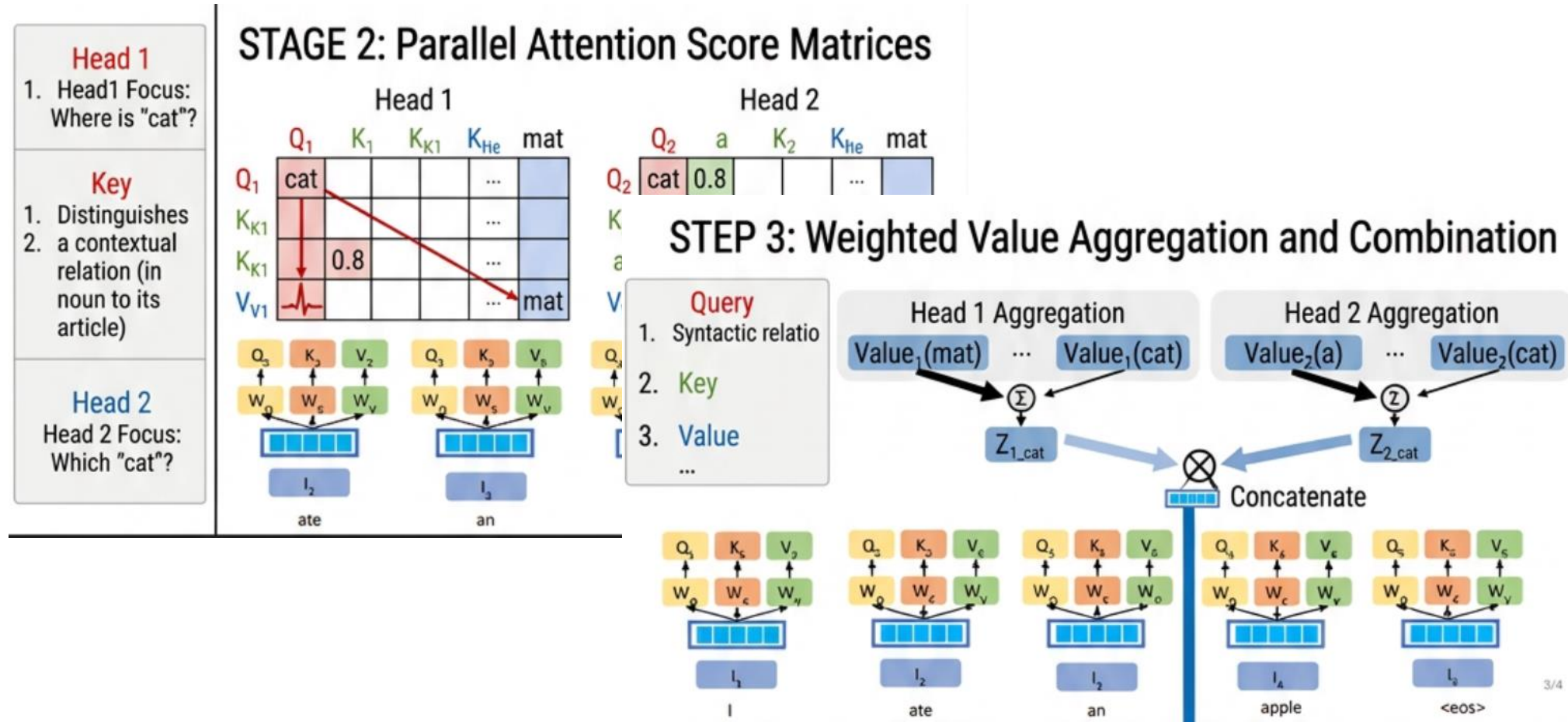
Transformer — Encoder – Multi Head Attention



Transformer — Encoder – Multi Head Attention

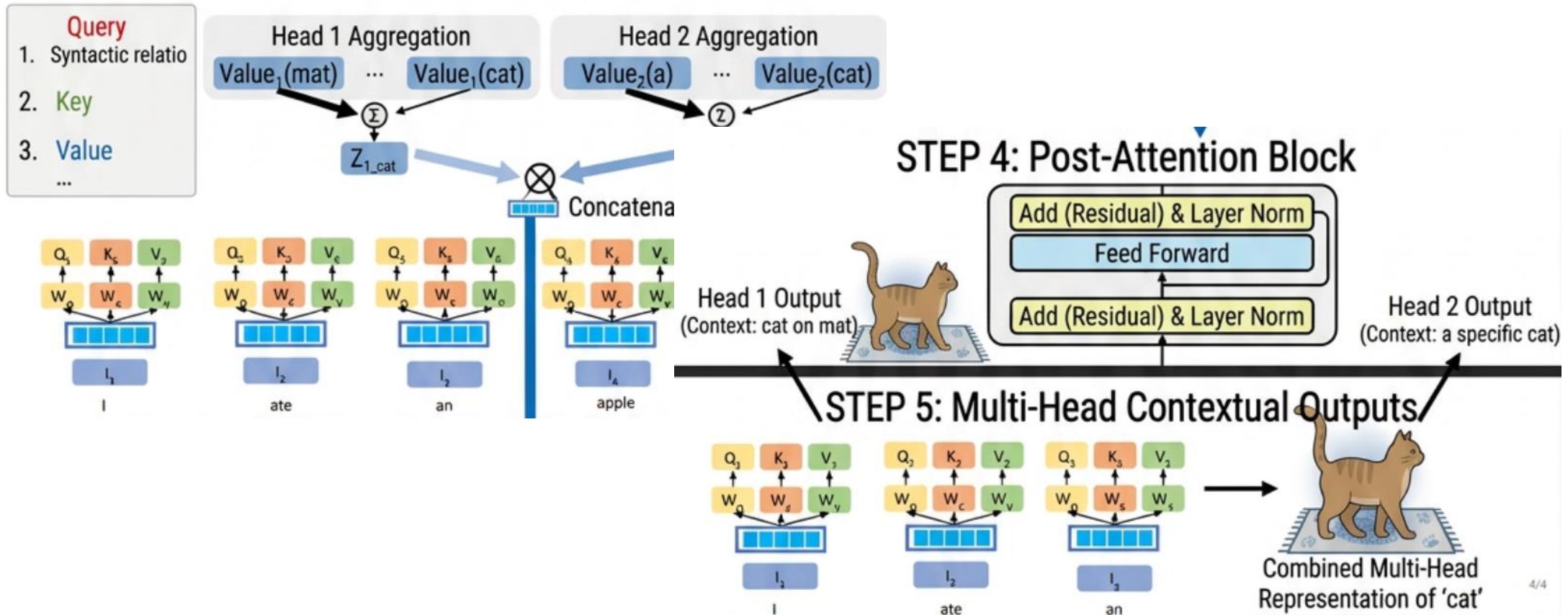


Transformer — Encoder – Multi Head Attention



Transformer — Encoder – Multi Head Attention

STEP 3: Weighted Value Aggregation and Combination



Transformer — Decoder

Each Decoder layer has **three** sub-layers (vs. two in Encoder)

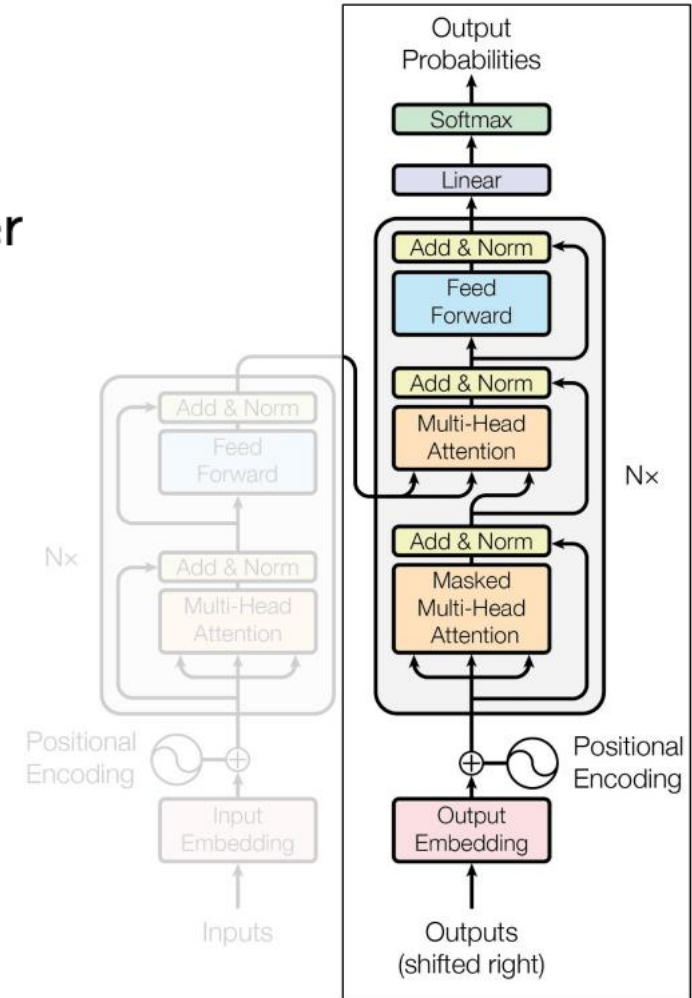
❶ **Masked Multi-Head Self-Attention:**

- Attend to previous tokens in the target sequence.
- *Constraint:* Cannot look at future tokens.

❷ **Encoder-Decoder Attention (Cross-Attention):**

- Query (Q) from Decoder.
- Key (K) & Value (V) from Encoder.

❸ **Feed Forward Network:** Same as Encoder.

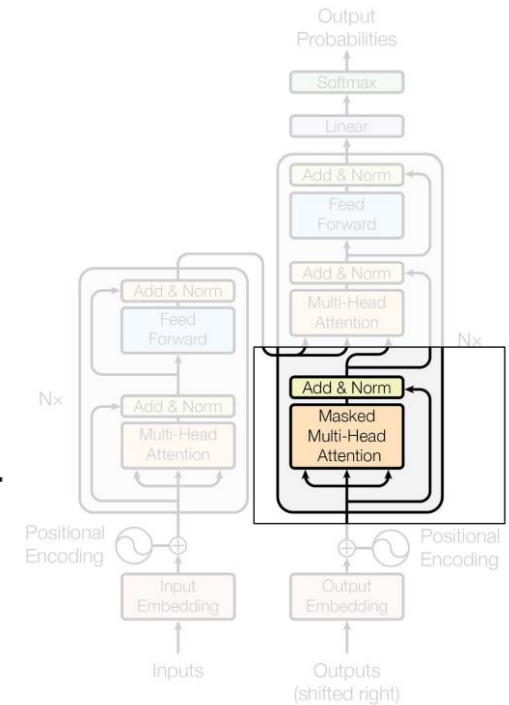


Transformer — Masked Self Attention

Scenario: We are training to translate "I am good" → "Je vais bien".

- Input to Decoder: <sos> Je vais bien
- At step $t = 1$ (Je), the model should predict vais.
- If we use standard Self-Attention, Je can "see" vais and bien.
- **Result:** The model learns to copy the next word rather than understanding syntax.

Solution: Apply a mask M such that $Attention(i, j) = 0$ if $j > i$.

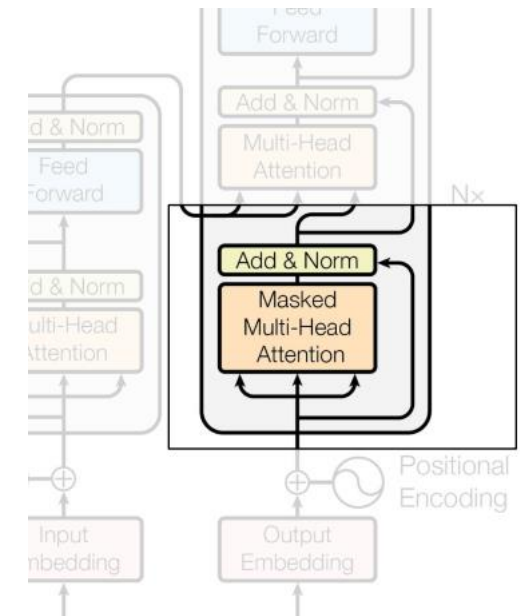


Transformer — Masked Self Attention

Let's calculate Masked Attention for a sequence of 3 tokens: A, B, C . Suppose $d_k = 1$ for simplicity.

Step 1: Compute $Q \cdot K^T$ Assume after linear projection, we have:

$$Q = \begin{bmatrix} 1 \\ 2 \\ 3 \end{bmatrix}, \quad K = \begin{bmatrix} 1 \\ 2 \\ 3 \end{bmatrix}$$
$$E = Q \cdot K^T = \begin{bmatrix} 1 \\ 2 \\ 3 \end{bmatrix} \begin{bmatrix} 1 & 2 & 3 \end{bmatrix} = \begin{bmatrix} 1 & 2 & 3 \\ 2 & 4 & 6 \\ 3 & 6 & 9 \end{bmatrix}$$



Transformer — Masked Self Attention

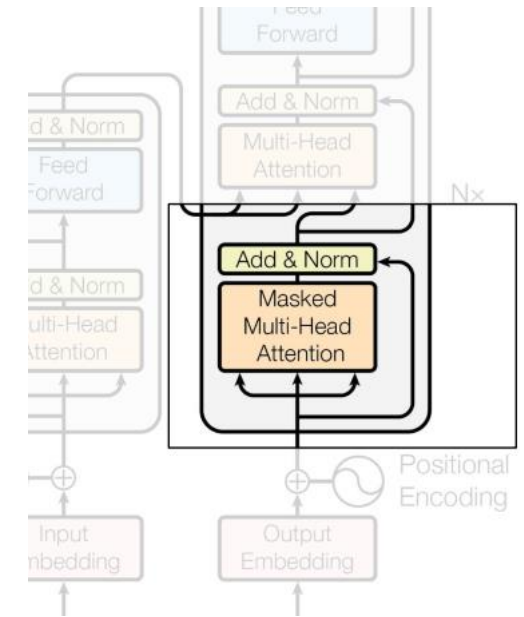
We create a **Mask Matrix** M of the same shape. Positions where $j > i$ (future) get $-\infty$.

$$M = \begin{bmatrix} 0 & -\infty & -\infty \\ 0 & 0 & -\infty \\ 0 & 0 & 0 \end{bmatrix}$$

$$\text{Masked Scores} = E + M$$

$$\begin{bmatrix} 1 & 2 & 3 \\ 2 & 4 & 6 \\ 3 & 6 & 9 \end{bmatrix} + \begin{bmatrix} 0 & -\infty & -\infty \\ 0 & 0 & -\infty \\ 0 & 0 & 0 \end{bmatrix} = \begin{bmatrix} 1 & -\infty & -\infty \\ 2 & 4 & -\infty \\ 3 & 6 & 9 \end{bmatrix}$$

Note: Adding $-\infty$ makes the value extremely small.



Transformer — Masked Self Attention

Apply Softmax row-wise: $\sigma(x)_i = \frac{e^{x_i}}{\sum e^{x_j}}$. Remember $e^{-\infty} \approx 0$.

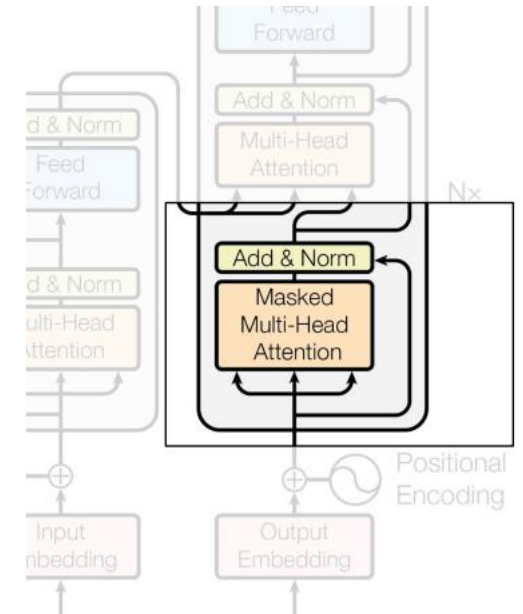
Row 1: $[1, -\infty, -\infty] \rightarrow [e^1, 0, 0]/e^1 \rightarrow [1.0, 0.0, 0.0]$

Row 2: $[2, 4, -\infty] \rightarrow [e^2, e^4, 0]/(e^2 + e^4) \rightarrow [0.12, 0.88, 0.0]$

Row 3: $[3, 6, 9] \rightarrow [e^3, e^6, e^9]/\sum \dots \rightarrow [0.002, 0.05, 0.95]$

$$A = \begin{bmatrix} 1.0 & 0.0 & 0.0 \\ 0.12 & 0.88 & 0.0 \\ 0.00 & 0.05 & 0.95 \end{bmatrix}$$

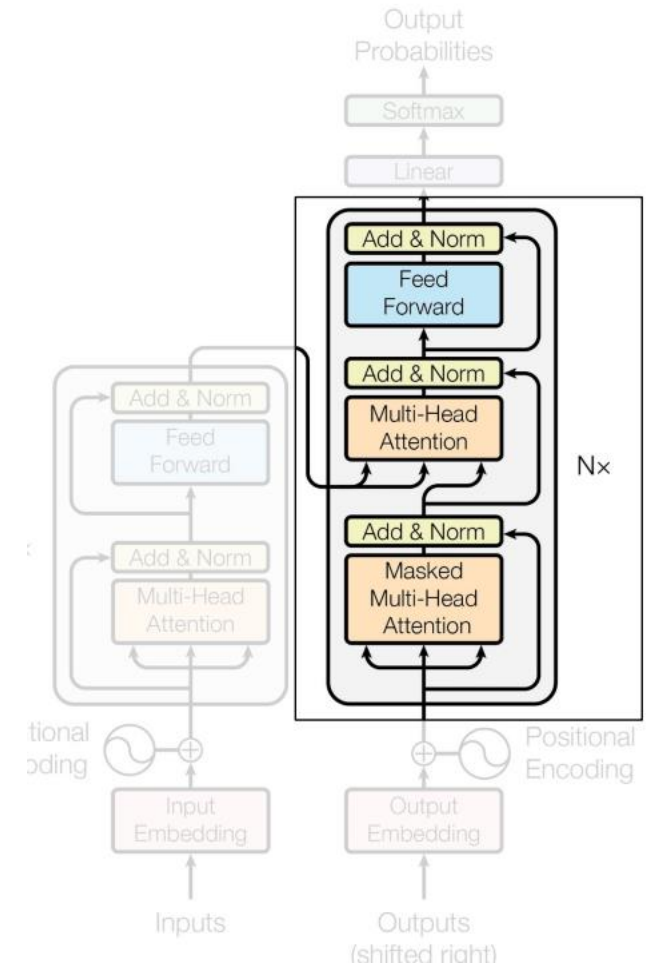
Result: The Upper Triangle is purely Zeros.



Transformer — Encoder Decoder Attention

This is where the magic of translation happens.

- We have the Decoder's current state (representing "I have generated X, Y").
- We have the Encoder's memory (representing the whole source sentence).
- **Goal:** Find which parts of the source are relevant *now*.



Transformer — Cross Attention

Scenario:

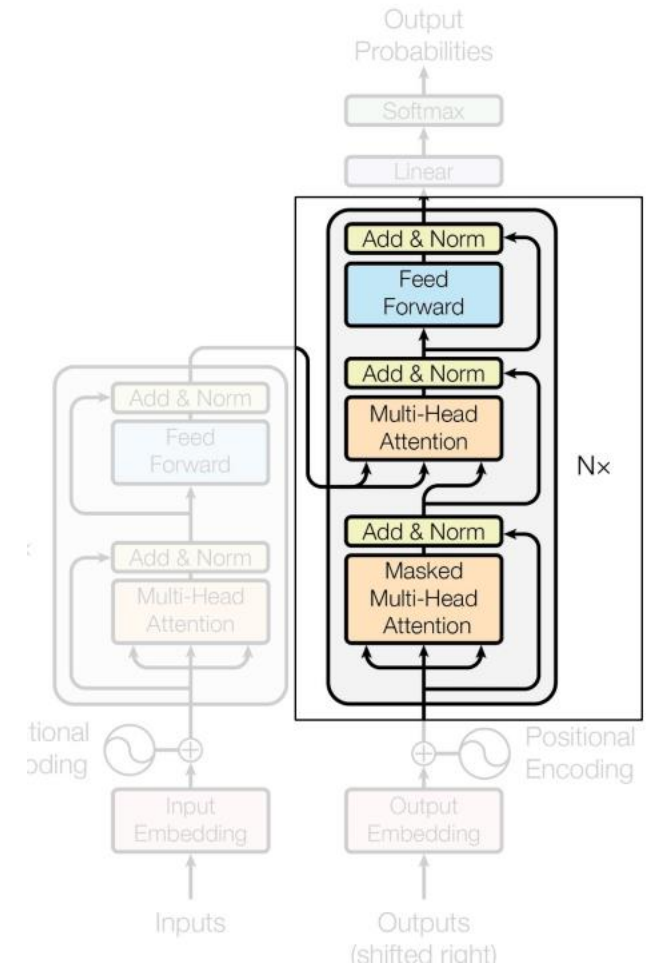
- **Encoder (Source):** "The cat" (2 tokens).
- **Decoder (Target):** Currently generating word after "Le" (1 token state).

Dimensions:

- Encoder Hidden States (H_{enc}): $[2 \times d_{model}]$
- Decoder Hidden State (H_{dec}): $[1 \times d_{model}]$

Let $d_k = 2$.

$$K_{enc} = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix} \text{ ("The", "cat")}, \quad Q_{dec} = [0 \quad 2] \text{ ("Le")}$$



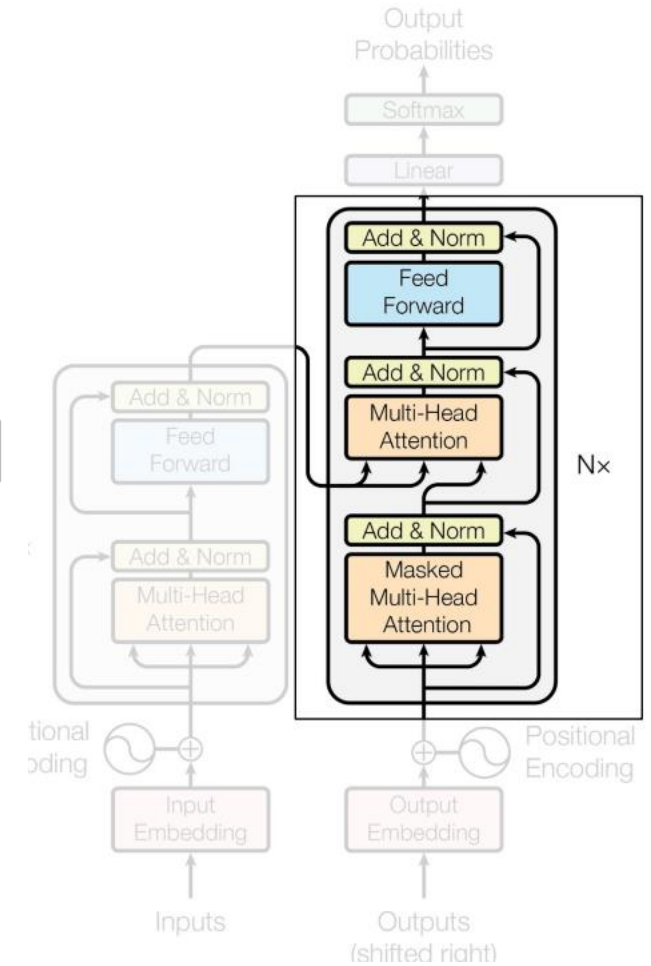
Transformer — Cross Attention

Calculate Scores ($Q \cdot K^T$):

$$\begin{aligned}\text{Score} &= Q_{dec} \cdot K_{enc}^T = \begin{bmatrix} 0 & 2 \end{bmatrix} \cdot \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}^T \\ &= \begin{bmatrix} 0 & 2 \end{bmatrix} \cdot \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix} = \begin{bmatrix} (0 \cdot 1 + 2 \cdot 0) & (0 \cdot 0 + 2 \cdot 1) \end{bmatrix} \\ &= \begin{bmatrix} 0 & 2 \end{bmatrix}\end{aligned}$$

Interpretation:

- Alignment with "The" (Word 1) = 0
- Alignment with "cat" (Word 2) = 2
- *The model focuses on "cat".*



Transformer — Cross Attention

Softmax & Value Aggregation:

$$\text{Softmax}([0, 2]) \approx [0.12, 0.88]$$

Let Values (V_{enc}) be same as Keys for simplicity:

$$V_{enc} = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}$$

$$\begin{aligned} \text{Context} &= [0.12, 0.88] \cdot \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix} \\ &= 0.12 \cdot [1, 0] + 0.88 \cdot [0, 1] = [0.12, 0.88] \end{aligned}$$

Result: The context vector passed to the next layer is 88% composed of the "cat" information.

Transformer — Cross Attention

After passing through N Decoder layers, we have a vector h_{final} for the current position.

Step 1: Linear Layer

- Projects from d_{model} (e.g., 512) to V_{vocab} (e.g., 30,000).
- Result: **Logits** (unnormalized scores).

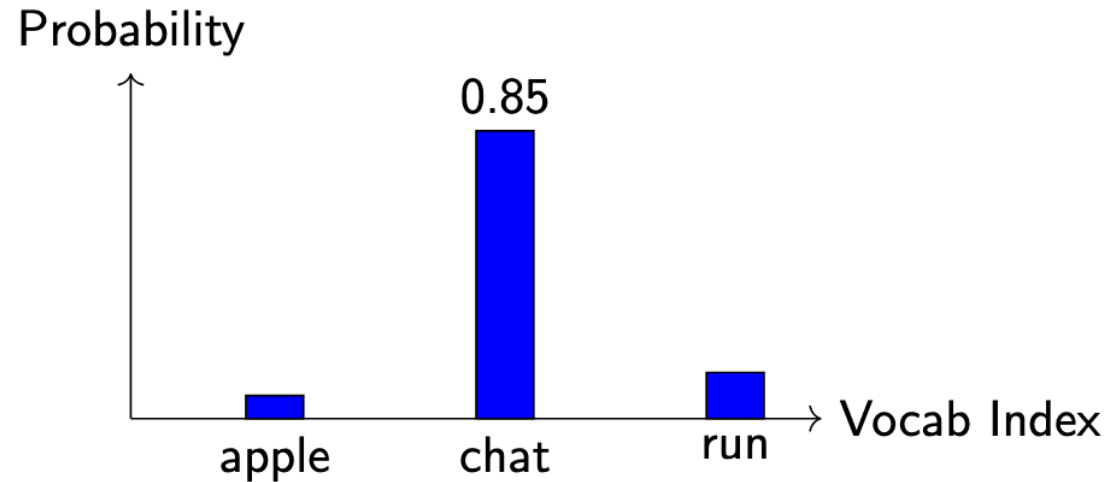
$$z = W \cdot h_{final} + b$$

$$[1 \times 30000] = [1 \times 512] \cdot [512 \times 30000]$$

Transformer — Cross Attention

Step 2: Softmax

- Converts logits into a probability distribution.
- Sum of all probabilities = 1.



Selection: Usually we pick the token with max probability (Greedy) or keep top-k paths (Beam Search).